UNIVERSITÀ DI PISA

Scuola di Ingegneria

Corso di Laurea Magistrale in Ingegneria Robotica
e dell'Automazione

Identificazione Sistemi Incerti

Identificazione veicolo a guida autonoma (AGV)

Professore:

R. Costanzi

A cura di:

Mansi Eligio

Nannola Marco

Scamarcio Aldo

Anno Accademico 2022/2023

Introduzione

Il seguente progetto ha come soggetto l'analisi di un veicolo industriale a guida Autonoma o (AGV) che si muove nello spazio di coordinate x ed y e dalla direzione theta.

L'AGV è dotato di tre ruote, due folli anteriori ed una motoruota posteriore attuata per l'angolo di sterzo ψ da una coppia τψ ed da una coppia relativa all'angolo di rotazione della motoruota al suo asse principale ϕ chiamata τϕ.

Nello scenario di modellizzazione sono presenti inoltre sensori di distanza e di direzione ciascuno posizionato rispettivamente nell'origine del sistema di riferimento e ad una distanza D lungo l'asse delle ascisse che ci serviranno per misurare l'orientazione dello sterzo PSI, la velocità di rotazione della ruota posteriore e l'angolo assegnato y-alpha.

Sarà ragionevole pensare che le misure ottenute dai sensori dovranno poi essere modellizzate con del rumore scelto in maniera appropriata a seconda del sensore, così come per il rumore di processo modellizzato sugli ingressi manipolabili.

Verranno discusse in maniera ragionevole diverse tematiche che riguardano la scelta di un insieme di parametri nominali che possano essere compatibili per lo scenario preso in considerazione e implementazione e modellazione del sistema fisico di cui faranno parte anche il modello dei sensori utilizzati.

Per una buona riuscita delle varie implementazioni sul Filtro di Kalman si definirà un modello di osservazione che ci permetterà di andare a esprimere le uscite dei nostri sensori in funzione delle variabili di stato.

Si implementerà, con l'ausilio di Matlab/Simulink, alcune delle tecniche di filtraggio viste a lezione tra cui il Filtro di Kalman Esteso (EKF) e in seguito l' Unscented Kalman Filter (UKF).

Per ciascuna delle tecniche di filtraggio si implementerà lo Smoother (RTS), un regolarizzatore, introdotto nel 1965 che prende il nome dagli inventori. La regolarizzazione è un interessante metodo offline che ci permetterà di andare a utilizzare anche le misure future per stimare lo stato corrente. In particolare, durante il filtraggio si avanza da 1 a K, mentre in fase di regolarizzazione si procede all'indietro da N a 1.

Infine si è creato uno script che permette di visualizzare in maniera chiara a seconda degli ingressi scelti, la traiettoria del nostro nostro veicolo in uno spazio 3D.

Per identificare il veicolo a guida autonoma abbiamo a disposizione il sistema fisico, il modello cinematico e il modello dinamico.

Capitolo 1

MODELLO DEL VEICOLO

Un veicolo, in questo caso un AGV, per quanto possa sembrar semplice, presenta particolari complessità legate alla presenza di componenti, quali trasmissione sterzo sospensioni pneumatici ecc. Perciò in tale progetto è stato sfruttato un modello semplificato, che verrà presentato in seguito. In particolare, in questo capitolo vengono trattati i due modelli legati, uno alla dinamica e uno alla cinematica del veicolo, rispettivamente, modello dinamico e modello cinematico e vengono descritti i diversi parametri annessi alle equazioni che identificano tali modelli.

1.1 Parametri del sistema

Prima di esplicitare le equazioni dei modelli è opportuno descrivere i parametri che le caratterizzano, gli stessi riportati nell'init del codice matlab allegato al presente documento.

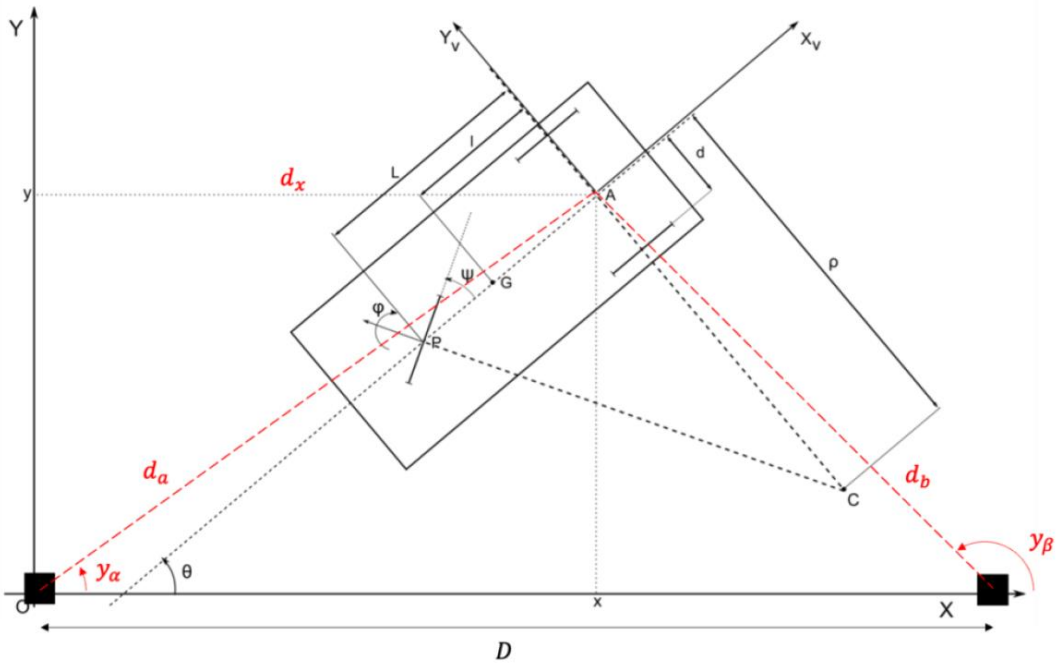


Figura 1. Modello grafico dell'AGV

- x_start = [0 0 0]'; vettore che rappresenta le coordinate x, y e z iniziali del sistema, ovvero identifica il punto di partenza di quest'ultimo.
- x_meas = [0 0 0]'; vettore che rappresenta le coordinate x, y e z iniziali del sistema, in questo caso misurate dai sensori applicati all'AGV
- L = 1.5; distanza in metri presente tra il punto A ovvero il punto medio dell'asse che lega la ruota destra anteriore a quella sinistra e il punto P dove è ancorata la ruota posteriore.

- rp = 0.2; ra=0.20; raggi delle tre ruote dell'AGV sempre definito in metri rispettivamente posteriore e anteriore
- l=0.8; distanza tra il punto A e il punto G raffigurante il centro di massa del mezzo
- IPy=0.045 e IPz=3.5; sono i due momenti d'inerzia del punto P quello rispetto all'asse y e quello all'asse z,
- Mv=80; Massa totale del mezzo
- mp=2; ma=2 massa delle ruote rispettivamente posteriori e anteriori
- IAy=0.025; IAz=7; momento d'inerzia del punto A rispetto all'asse y e z
- d=0.4; lunghezza semiasse gomme anteriori
- IGz=7.5; momento d'inerzia del punto G rispetto all'asse z

Infine, 3 costanti presenti nell'equazione dinamica del modello che vedremo successivamente:

- a = (Mv + mp + 2*ma + (2*IAy/(ra^2)));
- b = 1/2*(Mv*(l^2) + mp*(L^2) + 2*ma*(d^2) +IGz + IPz + 2*IAz + 2*IAy*((d/ra)^2));
- c = Mv*l + mp*L;

1.2 Modello dinamico

Al fine di analizzare le caratteristiche di un sistema (problema di analisi) è necessario procedere con l'identificazione, che consiste nel determinare il modello matematico che lega gli ingressi assegnati alle uscite note, tramite misure. Nel procedimento di analisi si cerca il modello matematico che più si avvicina al sistema reale, dove con modello matematico si intende la descrizione matematica delle caratteristiche dinamiche del sistema. Il modello matematico del sistema preso in considerazione nel nostro lavoro è descritto attraverso un insieme di equazioni differenziali nel dominio del tempo, un modello matematico con un giusto compromesso tra complessità e accuratezza; infatti, spesso non è conveniente appesantire troppo il modello per ottenere piccoli miglioramenti a costi elevati.

1.3 Equazioni del moto

$$\begin{cases} \tau_\varphi &= \left(ar_P^2\cos^2\psi + b\left(\frac{r_P}{L}\right)^2\sin^2\psi + I_{PY}\right)\ddot{\varphi} - I_{PZ}\frac{r_P}{L}\ddot{\psi}\sin\psi + \\ & \left(b\left(\frac{r_P}{L}\right)^2 - ar_P^2\right)\dot{\varphi}\dot{\psi}\sin\psi\cos\psi \\ \tau_\psi &= -I_{PZ}\frac{r_P}{L}\ddot{\varphi}\sin\psi + I_{PZ}\ddot{\psi} - I_{PZ}\frac{r_P}{L}\dot{\varphi}\dot{\psi}\cos\psi \end{cases}$$

In tali equazioni le principali semplificazioni che ricordiamo sono:

- l'angolo δ (angolo sterzo delle ruote anteriori) è nullo considerando solo l'azione di sterzatura della ruota posteriore;
- La massa del veicolo è concentrata nel baricentro fisso del sistema G;
- J è considerato il momento di inerzia attorno al baricentro G per l'asse z che passa verticalmente nel piano dove è situata l'autovettura;

Tali equazioni del moto sono state portate nella forma matriciale
$M\ddot{q} + N(\dot{q}, q) = \tau$ così da poter ricavare il vettore $\ddot{q}$ come $\ddot{q} = M^{-1}(\tau - N(\dot{q}, q)$

1.4 Modello CINEMATICO

Il modello cinematico è un modello che trascura completamente tutta la parte dinamica, per considerare solamente le relazioni cinematiche che legano le accelerazioni e le velocità relative al baricentro del veicolo nel piano xy del sistema di riferimento veicolo.

Può essere riassunto nel sistema a tre equazioni di seguito presentato:

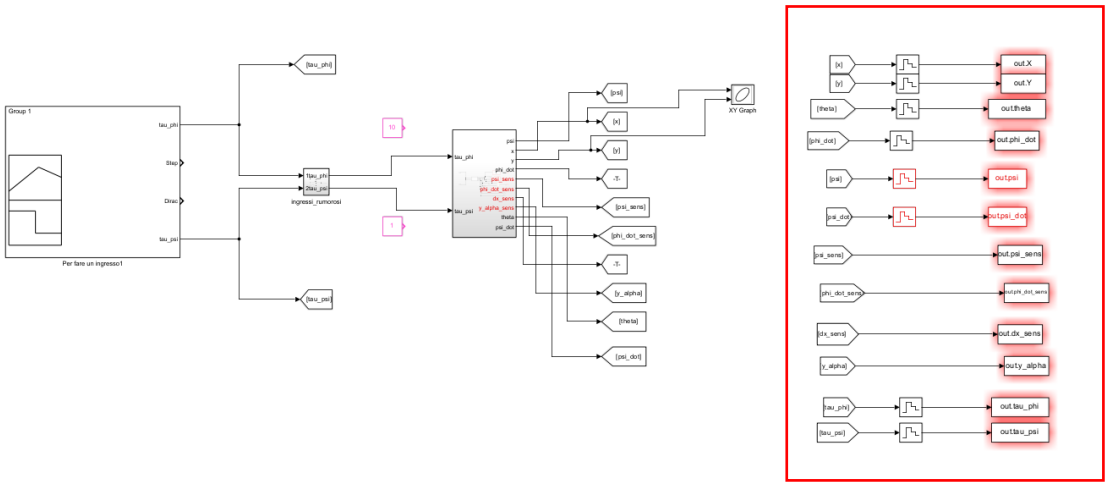
$$\begin{cases} \dot{x} &= r_P \dot{\varphi} \cos\theta \cos\psi \\ \dot{y} &= r_P \dot{\varphi} \sin\theta \cos\psi \\ \dot{\theta} &= -\frac{r_P \dot{\varphi} \sin\psi}{L} \end{cases}$$

Tale modello ha una grande importanza, infatti tramite la conoscenza delle variabili phi, psi_dot e theta ci permette di definire la stima esatta dello stato del sistema.

Ciò però è solo teoricamente possibile, in quanto il sistema è caratterizzato da rumore, errori che non sempre è possibile stimare e le accelerazioni non sono esattamente quelle del baricentro ovvero del punto G. A differenza del modello dinamico quello cinematico non presenta rilevanti semplificazioni e anch'esso è stato implementato in matlab tramite la Matlab Function.

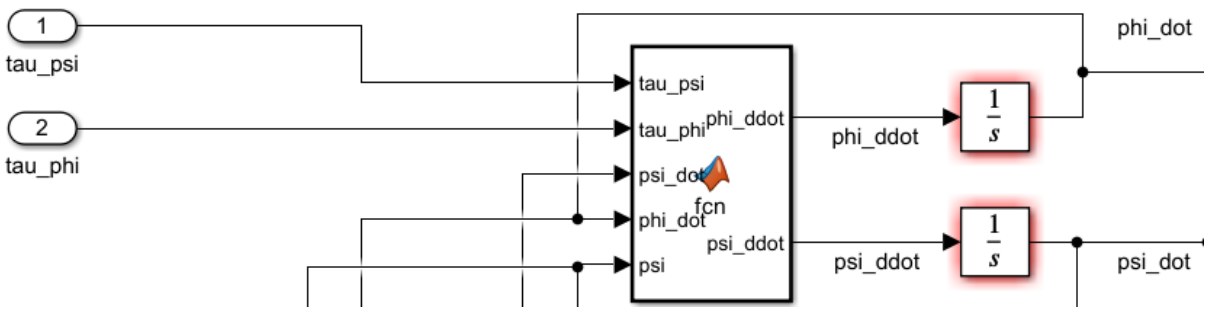
1.5 IMPLEMENTAZIONE DEL MODELLO SU SIMULINK

Nella figura (---) è rappresentato il modello simulink dell'intero sistema. Di seguito è spiegato il funzionamento di ogni singolo blocco, utile, insieme agli altri, alla verifica, analisi e identificazione del sistema.



1.5.1 Blocchi del modello

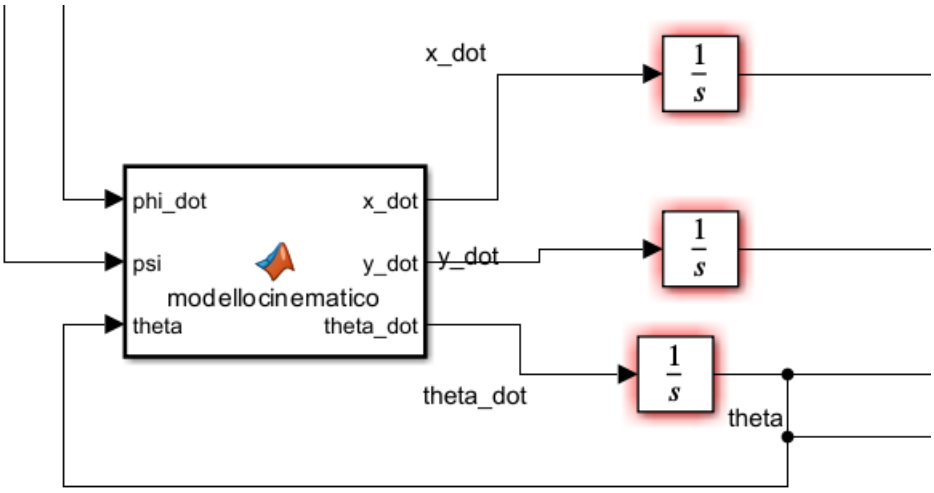
- Signal Builder: Il blocco Signal Builder consente di creare gruppi intercambiabili di sorgenti di segnali lineari a tratti e utilizzarli in un modello. Nella finestra è possibile creare segnali e definire le forme d'onda di uscita da quest'ultima. All'intero di tale blocco si sono definiti gli ingressi costanti, a gradino, rampa ecc utili, come vedremo successivamente, sia per la validazione del sistema che per la stessa simulazione.
- Ingressi rumorosi: Tale blocco è un sub-system che permette al modello, dovendo trattarne uno incerto, di avere in ingresso dei segnali precedentemente definiti con il signal builder sporcati da dei rumori limitati bianchi additivi.
- Modello dinamico: Avendo ottenuto in precedenza un sistema dinamico in forma matricale, questo, è facilmente implementabile in Simulink mediante il blocchetto MATLAB function:



Il modello appena descritto considera i due ingressi dinamici costituiti dalle coppie tau_psi (coppia applicata allo sterzo e responsabile della variazione dell'angolo psi) e tau_phi (coppia applicata alla ruota, responsabile della variazione dell'angolo phi, l'angolo di rotazione intorno all'asse della ruota). In uscita dal modello abbiamo le due rispettive accelerazioni phi_ddot e psi_ddot, da cui sarà possibile ricavare tramite la funzione integrator, la velocita e infine le posizioni. Proprio all'interno di questo blocco è stato inserito il modello in forma matriciale tramite comandi matlab.

```
function [phi_ddot,psi_ddot]= fcn(tau_psi,tau_phi,psi_dot,phi_dot,psi,a, b, c, rp, l, L, IPy, IPz, Mv, mp, ma, IAy, ra, d, IGz, IAz)
x = psi;

B = [a*rp^2*cos(x)^2+b*(rp/L)^2*sin(x)^2+IPy, -IPz*(rp/L)*sin(x); -IPz*(rp/L)*sin(x), IPz];
C = [(b*(rp/L)^2 - a*rp^2)*phi_dot*psi_dot*sin(x)*cos(x); -IPz*(rp/L)*phi_dot*psi_dot*cos(x)];
tau = [ tau_phi;tau_psi];

q=[phi_dot;psi_dot];

M = (inv(B)*(tau-C))';
phi_ddot= M(1);
psi_ddot= M(2);
```

- Modello cinematico:



Come possibile vedere dalla figura, il modello prende in ingresso le variabili phi_dot e psi derivanti dal modello dinamico e theta derivante dalla stessa relazione cinematica utilizzata nel modello. In uscita ci propone i tre stati del sistema, in particolare quest'ultimi vengono integrati per ottenere così la posizione del AGV nel piano cartesiano XY. A differenza del modello dinamico le equazioni presenti nella matlab function sono state inserite in forma non matriciale.

```
function [x_dot,y_dot,theta_dot] = modellocinematico(phi_dot,psi,theta,rp, L)

x_dot = rp*phi_dot*cos(theta)*cos(psi);
y_dot = rp*phi_dot*sin(theta)*cos(psi);
theta_dot = -(rp*phi_dot*sin(psi))/L;
```

- XYGraph: Il blocco Grafico XY traccia due segnali di ingresso su un grafico XY, plottando così la traiettoria del nostro veicolo.
- ToWhorkspace: L'ultimo blocco, circondato da un rettangolo rosso in figura(---), è stato un blocco fondamentale per il progetto. To Workspace, infatti, registra i dati connessi alla sua porta di input in un workspace da un modello Simulink. In genere, durante la simulazione, i flussi di dati registrati vengono trasmessi a Simulation Data Inspector e vengono restituiti nell'area di lavoro di base, ma solo quando la simulazione viene messa in pausa o interrotta. Tali dati poi sono utilizzati per la predizione e correzione nelle diverse tipologie di filtraggio, analizzate nel capitolo seguente. In particolare, prima di dare i segnali in uscita dal blocco principale del modello simulink al blocco ToWorkspace, questi sono stati prima campionati. Solo i segnali provenienti dai sensori non sono stati campionati in quanto lo sono già di natura.

1.6 Validazione

Affinché si valutassero i due modelli, si è deciso di effettuare delle simulazioni con particolari ingressi, in modo da ottenere delle traiettorie nel piano XY che andassero a confermare le traiettorie da noi predette. In particolare, si sono effettuate 7 simulazioni. La prima manovra consisteva nell'applicare i due segnali tau_phi e tau_psi costanti e pari a zero, aspettandoci che il sistema rimanesse fermo.

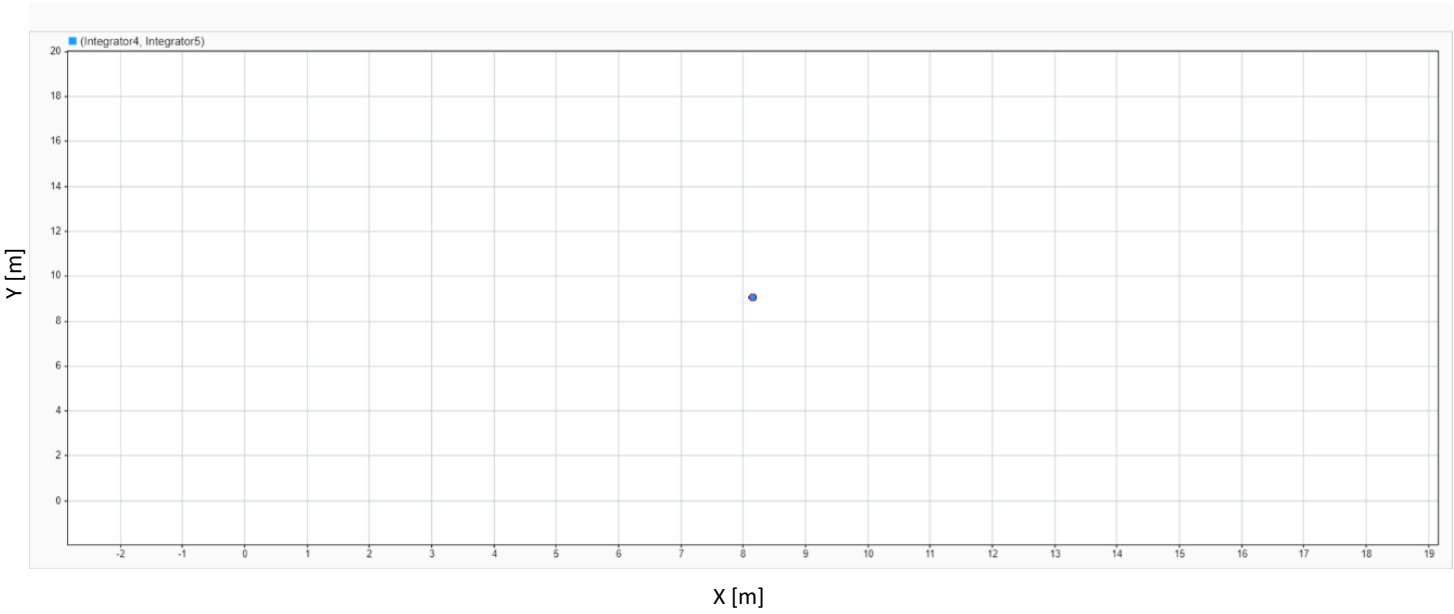


Figura: ingresso tau_phi nullo tau_psi nullo

Come illustrato in figura, infatti, effettuata la simulazione e quindi plottato la conseguente traiettoria del mezzo abbiamo validato questa ipotesi.

La seconda e la terza manovra avevano come obiettivo, quello di verificare il movimento lineare dell'AGV, rispettivamente l'avanzamento e la retroazione. Per entrambe le manovre si è applicata una tau_psi costante e pari a zero, invece per quanto riguarda l'altro ingresso tau_phi, si è optato nella seconda un tau_phi costante ma positiva, nella terza un tau_phi costante, ma negativa.

Ipotesi: il muletto nella seconda manovra si sposterà in avanti su una particolare linea retta, con un'inclinazione impostata dalla condizione iniziale dell'angolo di sterzo.

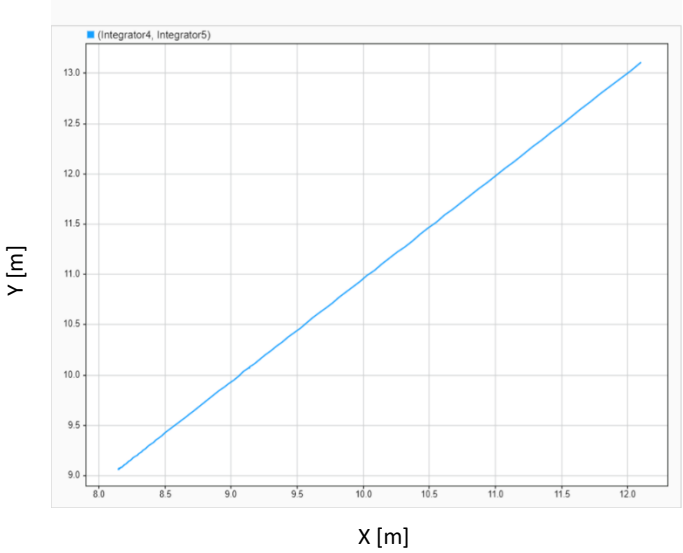


Figura: ingresso tau_phi positivo tau_psi nullo

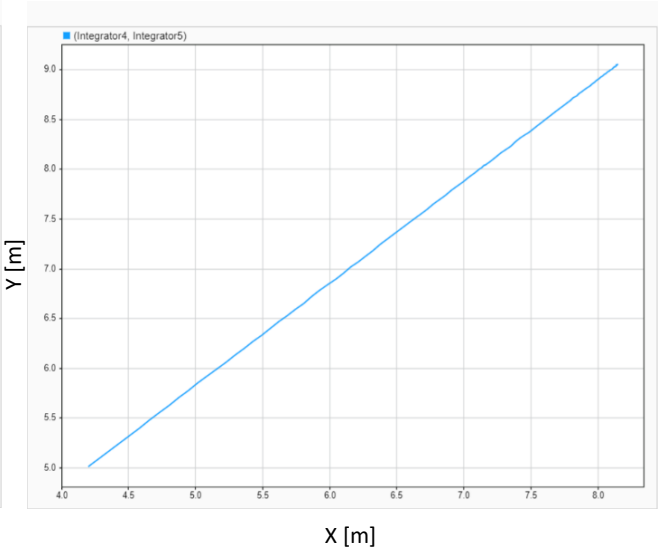


Figura: ingresso tau_phi negativo tau_psi nullo

La quarta e la quinta manovra conistevano nel verificare che applicando una tau_phi e una tau_psi costanti (ingresso a gradino), il mezzo percorresse una circonferenza o anche una ellisse senza convergenza ad un punto. In particolare nella quarta una tau_psi positiva e nella quinta tau_psi negativa, cosi da visionare in un caso la sterzata a destra e nell'altro una sterzata a sinistra. Anche questo, come visibile nelle immagini, ha restituito un feedback positivo.

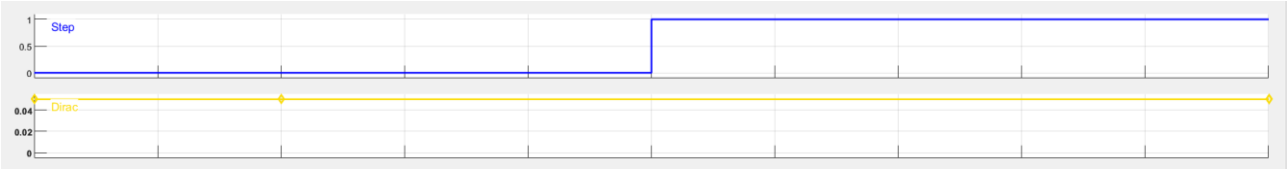


Figura: ingresso tau_phi positivo tau_psi positivo (gradino)

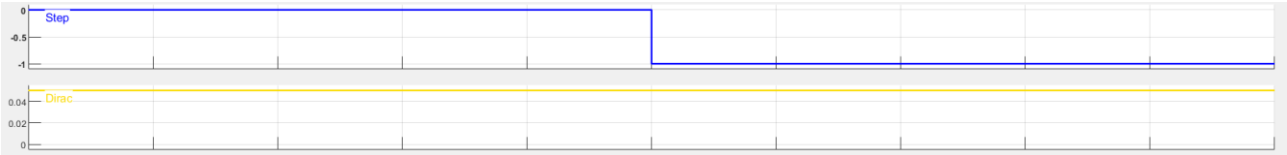
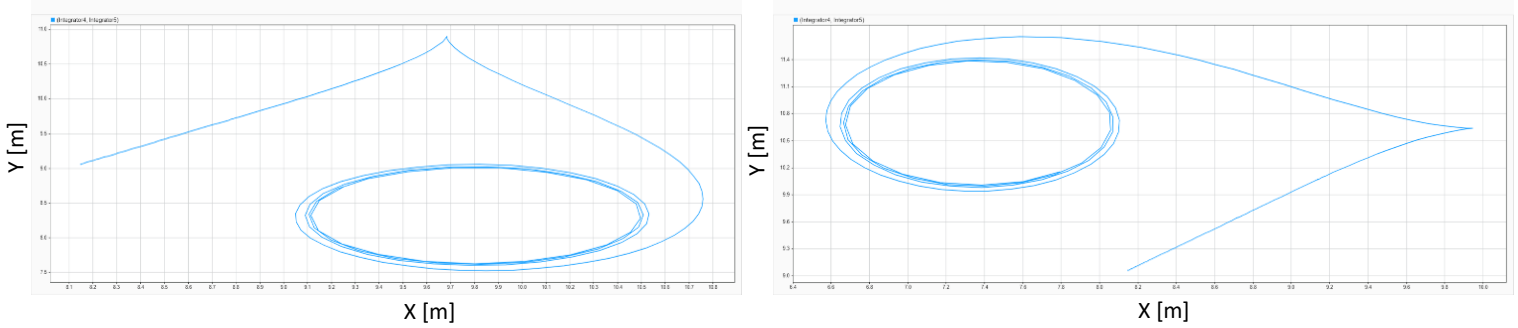


Figura: ingresso tau_phi positivo tau_psi negativo (gradino)



Infine, le ultime due simulazioni vedevano in input al sistema un ingresso a gradino per la tau_phi, quindi una coppia di accelerazione costante, e per la tau_psi una rampa, quindi una sterzata continua da parte del mezzo. Tale simulazione presenta dei risultati molto simili alle due precedenti, anche in questo caso si è analizzata la sterzata a destra e a sinistra, ma più precisamente si è validata la convergenza ad un punto della traiettoria. L'AGV percorre una spirale.

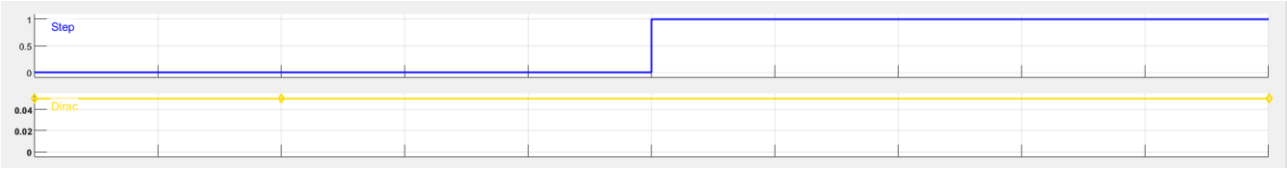


Figura: ingresso tau_phi positivo tau_psi positivo (rampa)

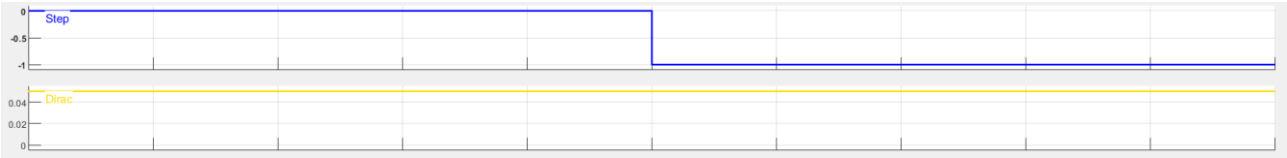
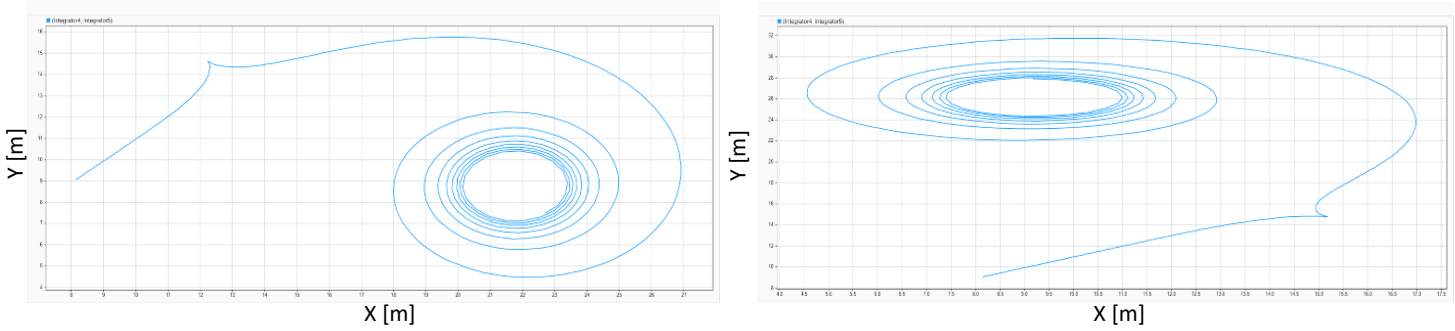


Figura: ingresso tau_phi positivo tau_psi negativo (rampa)

Grazie al blocco di integrazione disponibile in Simulink si è sfruttato lo stesso ingresso delle manovre precedenti per introdurre l'ingresso a rampa citato precedentemente



CAPITOLO 2

FILTRO DI KALMAN

Il filtro di Kalman è un filtro ricorsivo che permette di valutare lo stato di un sistema dinamico da una serie di misure che fuoriescono dai sensori e da un modello dinamico del sistema su cui il filtro fa riferimento. L'idea che c'è alla base è quella di migliorare le stime fornite dal modello matematico mediante le misure in uscita dai sensori. Pertanto, tale procedimento avviene ad ogni passo di campionamento cosicché l'errore tra la variabile stimata e la variabile vera converga il più velocemente possibile a zero, stimando lo stato perfettamente.

Ricapitolando, le fasi principali di cui si compone il filtro di Kalman sono due:

-PREDIZIONE: si sfrutta il modello matematico del sistema ricavato a partire dal modello dinamico e cinematico per fare una predizione dello stato al passo corrente

-CORREZIONE: mediante le misure in uscita dai sensori si corregge lo stato predetto. Naturalmente le misure sono affette da errori poiché i sensori sono degli strumenti con rumori di misura.

La prima cosa da fare quando si vuole implementare un filtro di Kalman è definire lo stato da stimare. Nel caso di interesse è stato scelto il seguente vettore di stato:

X (1) = x; X(2) = y; X(3) = theta; X(4) = phi_dot; X(5) = psi; X(6) = psi_dot;

Le prime tre componenti del vettore di stato individuano la posizione dell'AGV nel piano xy e l'angolo di orientazione dell'AGV rispetto all'origine degli assi (theta).

Phi_dot rappresenta la velocità di rotazione della ruota posteriore che determina la velocità di movimento dell'AGV. Psi rappresenta l'angolo di sterzo della ruota posteriore ed invece psi_dot la velocità con cui varia l'angolo di sterzo. Tale vettore di stato è stato inizializzato in Matlab come una variabile simbolica. Insieme agli stati sono fondamentali gli ingressi controllabili del sistema tau_phi e tau_psi i quali rappresentano rispettivamente le coppie da applicare per muovere e sterzare l'AGV. Anch'essi sono stati inizializzati in Matlab come variabili simboliche. Tale scelta è stata fatta poichè cosi è possibile derivare gli stati con semplicità.

Dopo aver definito le variabili di stato è necessario assegnare delle condizioni iniziali a tali stati. Per il sistema in analisi si è supposto che non si conoscesse perfettamente la posizione iniziale e l'orientazione dell'AGV pertanto è stata supposta random, inoltre si è supposto che inizialmente l'AGV fosse fermo (phi_dot= psi_dot = 0) con angolo di sterzo iniziale anch'esso random. Si è supposto che l'AGV si muova in un quadrato di dimensione 10x10m, quindi la posizione è random in questo quadrato. Non conoscendo la posizione iniziale dell'AGV è necessario quindi definire un disco di incertezza con il quale con grande probabilità i valori degli stati dell'AGV ricadano dentro. Tali dischi di incertezza sono stati scelti piccoli e in modo tale da rispettare la realtà.

```
std_dev_init_x_y = 0.5 ;%supponendo disco di incertezza 0.5
std_dev_init_angoli = 0.1 ; %supponendo varianza di 0.6 gradi
std_dev_init_vel_ang = 0.05 ; %supponendo varianza di 0.15 rad/s
xr_0 = x_0 + std_dev_init_x_y*randn(1); %posizione x iniziale sporcata
yr_0 = y_0 + std_dev_init_x_y*randn(1); %posizione y iniziale sporcata
thetar_0 = theta_0 + std_dev_init_angoli* randn(1); %posizione theta iniziale sporcata
psir_0 = psi_0 + std_dev_init_angoli*randn(1); %posizione psi iniziale sporcata
```

Le deviazioni standard rappresentano proprio i dischi di incertezza i quali vengono costruiti mediante una distribuzione normale (in Matlab con comando randn). In questo modo non conosciamo perfettamente la posizione dell'AGV ma sappiamo che si può trovare con grande probabilità in una zona definita dal disco di incertezza.

Le informazioni sullo stato iniziale sono state inserite tutte in un unico vettore denominato X_hat ed invece le informazioni sulle varianze (deviazioni standard al quadrato) sono contenute nella matrice diagonale P di dimensione 6x6. Tali matrici saranno fondamentali nell'implementazione del filtro.

Un altro passo fondamentale è definire le caratteristiche dei sensori, in particolare le deviazioni standard poiché si è detto che i sensori sono strumenti affetti da rumore di misura. I sensori assegnati misurano le seguenti grandezze:

- Posizione dello sterzo (psi)
- Velocità di rotazione della ruota posteriore (phi_dot)
- Distanza da asse y (dx)
- Angolo y-alpha

Per il caso in esame sono stati scegli come sensori degli encoder per la misura delle velocità angolari e per la misura degli angoli. Le deviazioni standard di tali strumenti sono state scelte in modo tali da avvicinarsi il più possibile alla realtà e inserite tutte in una matrice diagonale denominata R.

Per la modellazione del rumore di misura si è scelto di utilizzare un vettore simbolico V di quattro elementi utile successivamente per la costruzione del modello di osservazione. Stessa cosa per il rumore di processo sugli ingressi manipolabili, esso è costituito da un vettore simbolico di due elementi e le varianze di tale rumore inserite in una matrice diagonale chiamata Q.

Come già preannunciato in precedenza per l'implementazione del filtro è necessario costruire la matrice del modello chiamata f a partire dalle equazioni cinematiche e dinamiche. In precedenza abbiamo visto che il modello dinamico può essere scritto nella forma B*q_ddot + C(q,q_dot) = tau. Ora siccome abbiamo le variabili di stato scritte in simbolico possiamo sostituirle nelle equazioni. Nel modello inoltre dobbiamo anche tener conto del rumore del segnale di ingresso ecco perché alla tao si sommano le W definite in precedenza come disturbo di processo. Così è stato ricavato il vettore di stato derivato scritto in funzione delle equazioni. A questo punto è necessario discretizzare mediante la formula X_k+1 = X_k + dt* X_dot ottenendo così la f. Essa sarà utile nell'implementazione del filtro di Kalman esteso, in particolare nella parte di predizione.

```
B = [a*rp^2*cos(X(5))^2+b*(rp/L)^2*sin(X(5))^2+IPy, -IPz*(rp/L)*sin(X(5)); -IPz*(rp/L)*sin(X(5)), IPz];


C = [(b*(rp/L)^2-a*rp^2)*X(4)*X(6)*sin(X(5))*cos(X(5)); -IPz*(rp/L)*X(4)*X(6)*cos(X(5))];
tau_w = [tau(1)+ W(1) tau(2)+W(2)] ;

M = (inv(B)*((tau_w)' - C ));

%Calcolo X_dot

x1_dot=rp*X(4)*cos(X(3))*cos(X(5));
x2_dot=rp*X(4)*sin(X(3))*cos(X(5));
x3_dot=-(rp*X(4)*sin(X(5)))/L;
x4_dot=M(1); %phi_ddot
x5_dot=X(6); %psi_dot
x6_dot=M(2); %psi_ddot
```

```
dt=t_sample;% tempo di campionamento
f=X+dt*X_dot;
```

Insieme alla f, è utile definire il modello di osservazione che servirà nella parte di Innovazione del filtro. Per il calcolo di h basta costruire un vettore dove esprimiamo le uscite dei sensori in funzioni delle variabili di stato. Ogni riga del vettore corrisponde ad un sensore a cui ci aggiungiamo del rumore di misura costruito precedentemente con la variabile simbolica V.

```
y_alpha = atan2(X(2),X(1));
h = [X(5) + V(1) , X(4) + V(2) , X(1) + V(3) ,  y_alpha + V(4) ];
```

Dopo la valutazione del modello di osservazione e del modello del sistema, si passa all'implementazione del filtro di Kalman esteso che sfrutta proprio entrambi i modelli con il calcolo delle matrici del linearizzato.

2.1 EXTENDED KALMAN FILTER

Il filtro di Kalman esteso è un tipo di filtro che sfrutta l'approssimazione al primo ordine dello sviluppo in serie di Taylor. Il primo passo per l'implementazione di tale filtro è il calcolo delle matrici del linearizzato mediante lo Jacobiano. Sono 4 le matrici da calcolare:

- F : essa si calcola derivando la f (modello del sistema) per gli stati, di conseguenza sarà una matrice 6x6
- D : essa si calcola derivando la f per il rumore applicato agli ingressi, di conseguenza sarà una matrice 6x2 ed inoltre siccome le uniche due righe con rumore sono la 4 e la 6, esse saranno le uniche righe diverse da 0
- H : essa è una matrice che si calcola derivando h (modello di osservazione) per gli stati , perciò sarà una matrice 4x6
- Mk : essa si calcola derivando la h per gli ingressi rumorosi ai sensori, perciò è una matrice 4x4. Siccome gli ingressi sono additivi, essa è una matrice identità.

Per calcolare queste matrici in Matlab è stata sfruttata la funzione jacobian che prende in input la funzione da derivare e in cosa derivarla.

```
F=jacobian(f,X);

D=jacobian(f,W);


H=jacobian(h,X);

Mk=iacobian(h,V);
```

Come si vedrà successivamente, per completare la linearizzazione le prime due matrici saranno valutate in x_predetto al passo k con disturbi di processo nulli , invece le altre due matrici valutate nella stima più recente che ho a disposizione e con rumore di sensore nullo.

Tutti i calcoli fatti finora è possibile trovarli nello script ***init.mat.***

Dopo aver visto i passi preliminari per l'inizializzazione del filtro di Kalman esteso, si passa all'implementazione del filtro in sé con le parti di predizione ed innovazione. Tali passaggi è possibile trovarli nello script ***EKF.mat.***

Per prima cosa è stato necessario inserire tutti i dati che fuoriescono da Simulink in una struttura chiamata ***log_vars*** poiché di facile collezione. Per fare questo si è sfruttato il blocco ***To Workspace*** in Simulink cosi che i dati vengono trasferiti al workspace di Matlab. I dati in questione sono i seguenti:

- log_vars = [];
- log_vars.X = (out.X)'; coordinata vera X dell'AGV
- log_vars.Y = (out.Y)'; coordinata vera Y dell'AGV
- log_vars.dx_sens = (out.dx_sens)'; grandezza dx in uscita dal sensore (affetta da rumore)
- log_vars.phi_dot = (out.phi_dot)'; grandezza phi_dot vera dell'AGV
- log_vars.phi_dot_sens = (out.phi_dot_sens)'; grandezza phi_dot in uscita dal sensore
- log_vars.psi = (out.psi)'; grandezza psi vera dell'AGV
- log_vars.psi_dot= (out.psi_dot)'; grandezza psi_dot vera dell'AGV

- log_vars.psi_sens = (out.psi_sens)'; grandezza psi in uscita dal sensore
- log_vars.tau_phi = (out.tau_phi)'; ingresso applicato all'AGV senza rumore
- log_vars.tau_psi = (out.tau_psi)'; ingresso applicato all'AGV senza rumore
- log_vars.theta = (out.theta)'; grandezza theta vera dell'AGV
- log_vars.y_alpha = (out.y_alpha)'; grandezza y_alpha in uscita dal sensore
- log_vars.x_hat = x_hat; vettore iniziale dello stato visto al paragrafo precedente
- log_vars.P = P; matrice P iniziale delle varianze

Tutti questi valori sono stati elaborati dal blocco Simulink visto al capitolo precedente, campionati e portati in Matlab. Gli ingressi manipolabili e i dati in uscita dai sensori servono nella fase di predizione e correzione invece tutti gli altri servono per verificare che il filtro alla fine funzioni mediante un grafico che valuti l'errore tra la variabile vera e quella stimata dal filtro.

Adesso si passa al corpo principale dell'EKF con il ciclo ***for*** che contiene sia la parte di predizione che di correzione mediante due ***MatlabFunction*** chiamate ***predict_EKF*** e ***correct_EKF.*** Il ciclo for scorre ogni dt che è il tempo di campionamento del filtro che in questa situazione è uguale al tempo di campionamento dei sensori, fino al tempo di simulazione.

```
k = 0;
for t = 0:dt:t_simulation
    k = k + 1;

    [x_hat,P] = predict_EKF(x_hat,P,Q,dt,log_vars.tau_phi(k),log_vars.tau_psi(k),W,F,D,X_dot,ht,ht1,ht2);
    [x_hat,P,e] = correct_EKF(x_hat, P, R,log_vars.dx_sens(k),log_vars.phi_dot_sens(k),log_vars.psi_sens(k),log_vars.y_alpha(k),ht3,ht4,Mk,h,H);
 log_EKF.x_hat(k,1:6) = x_hat;
 log_EKF.P(k,:,:) = P;
 log_EKF.e(k,1:4) = e;

end
```

È stata creata un'altra struttura log_EKF per salvare x_hat, P e l'errore.

PREDIZIONE: nella parte di predizione si passa da x^k/k a x^k+1/k. Questo step dipende esclusivamente dal modello del sistema e quello che si fa è valutare le matrici del linearizzato F,D sostituendone il vettore x^k/k, gli ingressi manipolabili tau_phi e tau_psi al passo k in uscita da Simulink ed il valore del rumore di processo posto uguale a 0. Per effettuare una sostituzione nelle matrici simboliche F.D e X_dot si è usata la ***MatlabFunction*** e il comando ***feval.*** Poi per effettuare

la predizione x^k+1/k ed aggiornare la matrice P si sfrutta le formule viste dalla teoria e contenute nell'immagine successiva.

```
%predizione

function  [x_hat,P] = predict_EKF(x_hat,P,Q,dt,tau_phi,tau_psi,W,F,D,X_dot,ht,ht1,ht2)

F = feval(ht,x_hat(3),x_hat(4),x_hat(5),x_hat(6),tau_phi,tau_psi,0,0);
D = feval(ht1,x_hat(5));
X_dot = feval(ht2,x_hat(3),x_hat(4),x_hat(5),x_hat(6),tau_phi,tau_psi,0,0);
 x_hat = x_hat + X_dot * dt ;
   P = F*P*F' + D*Q*D';
end
```

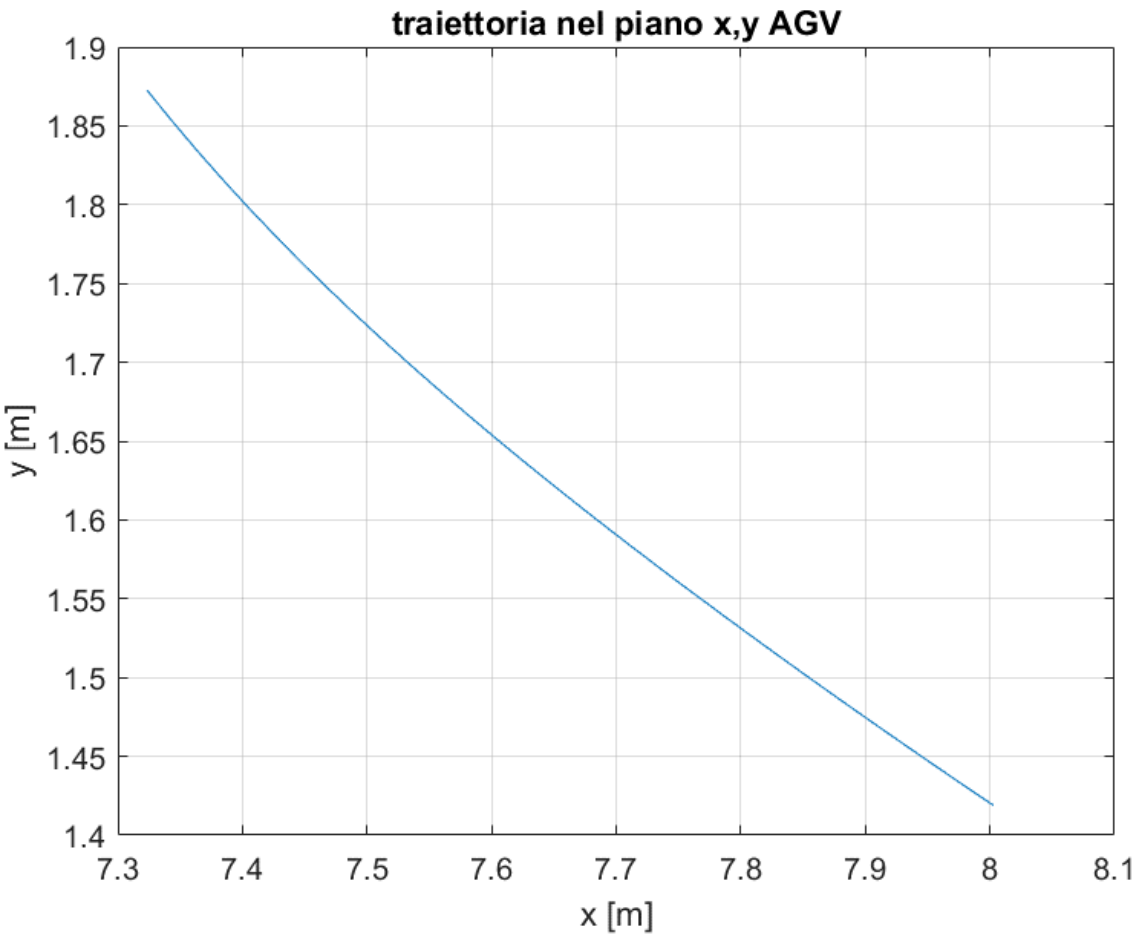
CORREZIONE: nella parte di correzione si passa da x^k/k-1 a x^k/k. Questa fase dipende esclusivamente dalle misure che escono dai sensori infatti quello che si fa è correggere la misura predetta mediante le osservazioni. In questa fase quindi serve la matrice h (modello di osservazione) valutata in x_hat (calcolato nella predizione) con rumori di misura nulli ed inoltre le matrici H e Mk valutate anch'esse in x-hat. Poi la valutazione di x_hat e P è schematica e vista nella teoria. Affinché gli angoli misurati dai sensori siano compresi tra -pi e pi è stata sfruttata la funzione atan2.

```
%Correzione

 function [x_hat,P,e] = correct_EKF(x_hat, P, R,dx_sens,phi_dot_sens,psi_sens,y_alpha,ht3,ht4,Mk,h,H)
 H = feval(ht3,x_hat(1),x_hat(2));
 h = feval(ht4,x_hat(1),x_hat(2),x_hat(4),x_hat(5),0,0,0,0);
 e = [psi_sens;phi_dot_sens;dx_sens;y_alpha] - h';
 e(1) = atan2(sin(e(1)), cos(e(2)));
 e(3) = atan2(sin(e(3)), cos(e(3)));
 e(4) = atan2(sin(e(4)),cos(e(4)));
 S = vpa(H*P*H' + Mk*R*Mk',3);
 G = vpa(P*H'*inv(S),3);
 x_hat = x_hat + G*e
 P = (eye(6) - G*H)*P*(eye(6) - G*H)' + G*Mk*R*Mk'*G'
  end
```

Per vedere che il filtro funziona bene è stata fatta una simulazione. La simulazione dura 10 secondi e di seguito sono riportati i grafici che mostrano la correttezza del filtro.

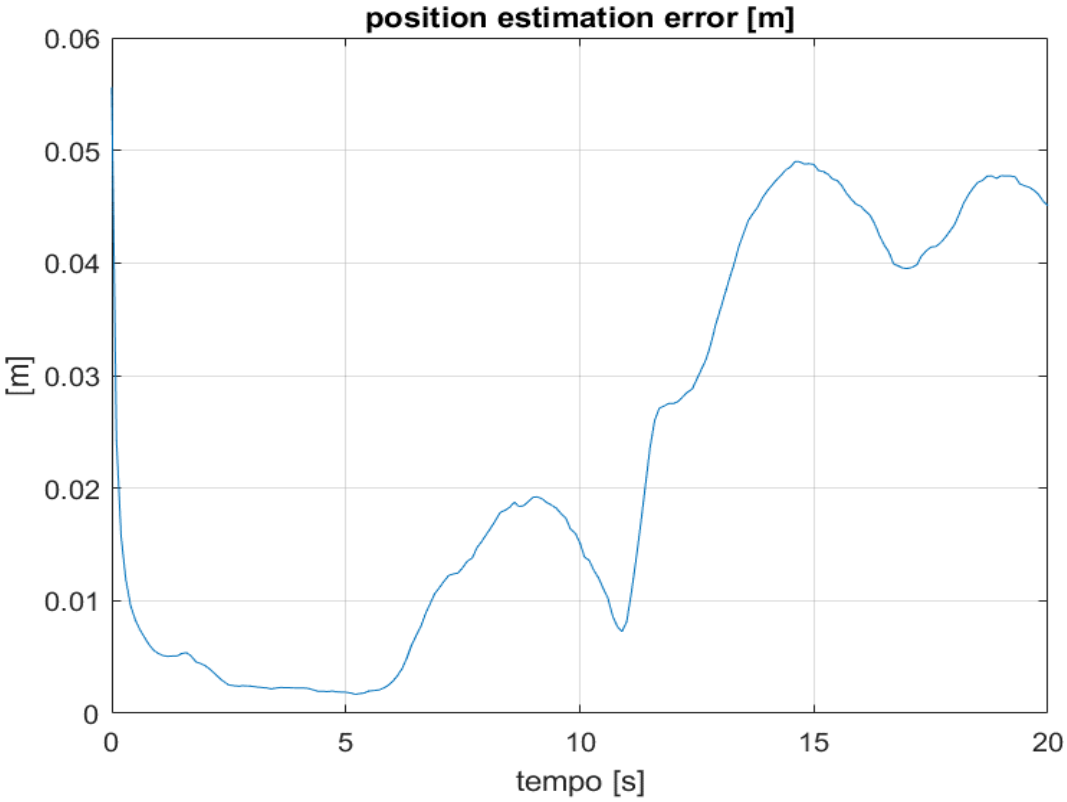
Gli ingressi applicati sono i seguenti: lo step per la tau_phi e l'impulso integrato per la tau_psi

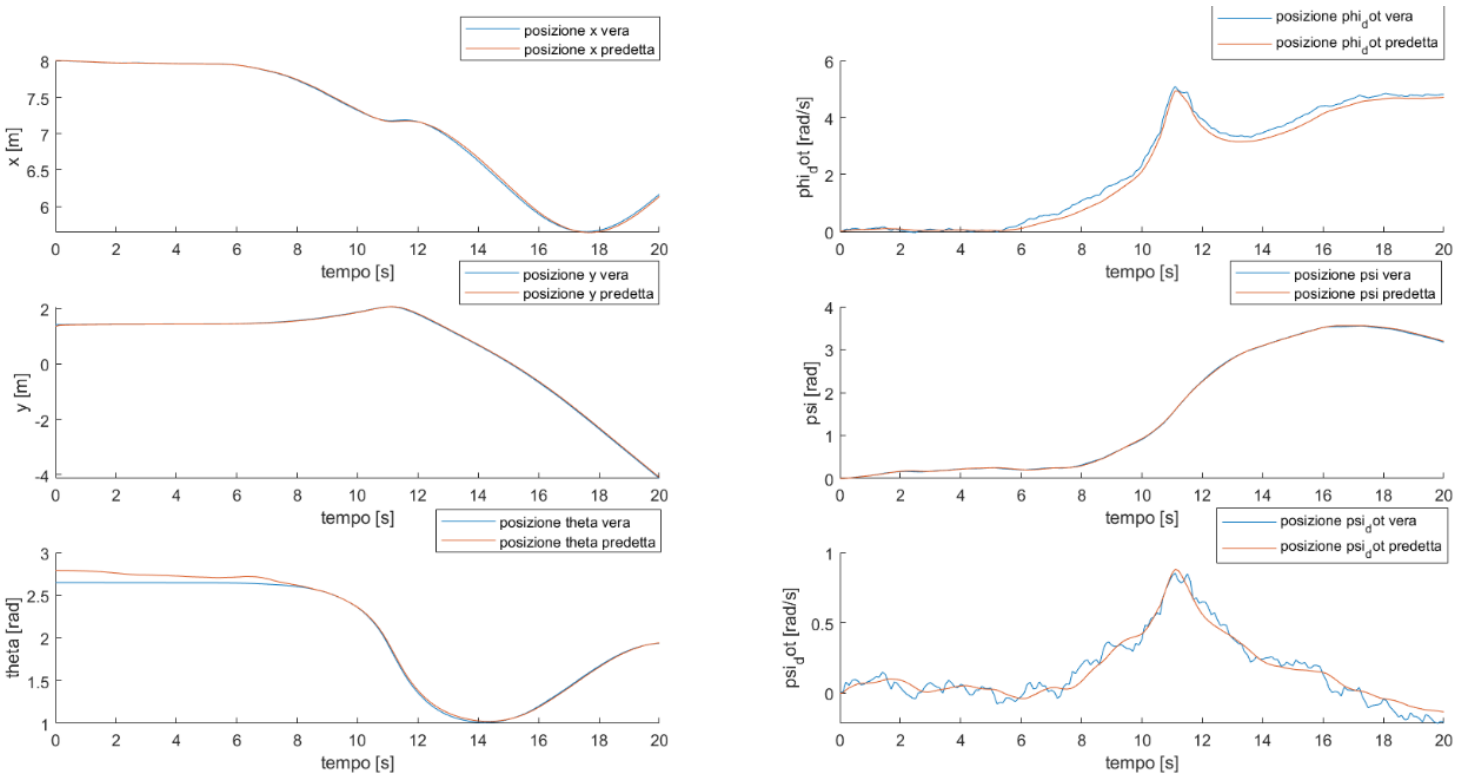
La traiettoria dell'AGV è la seguente:

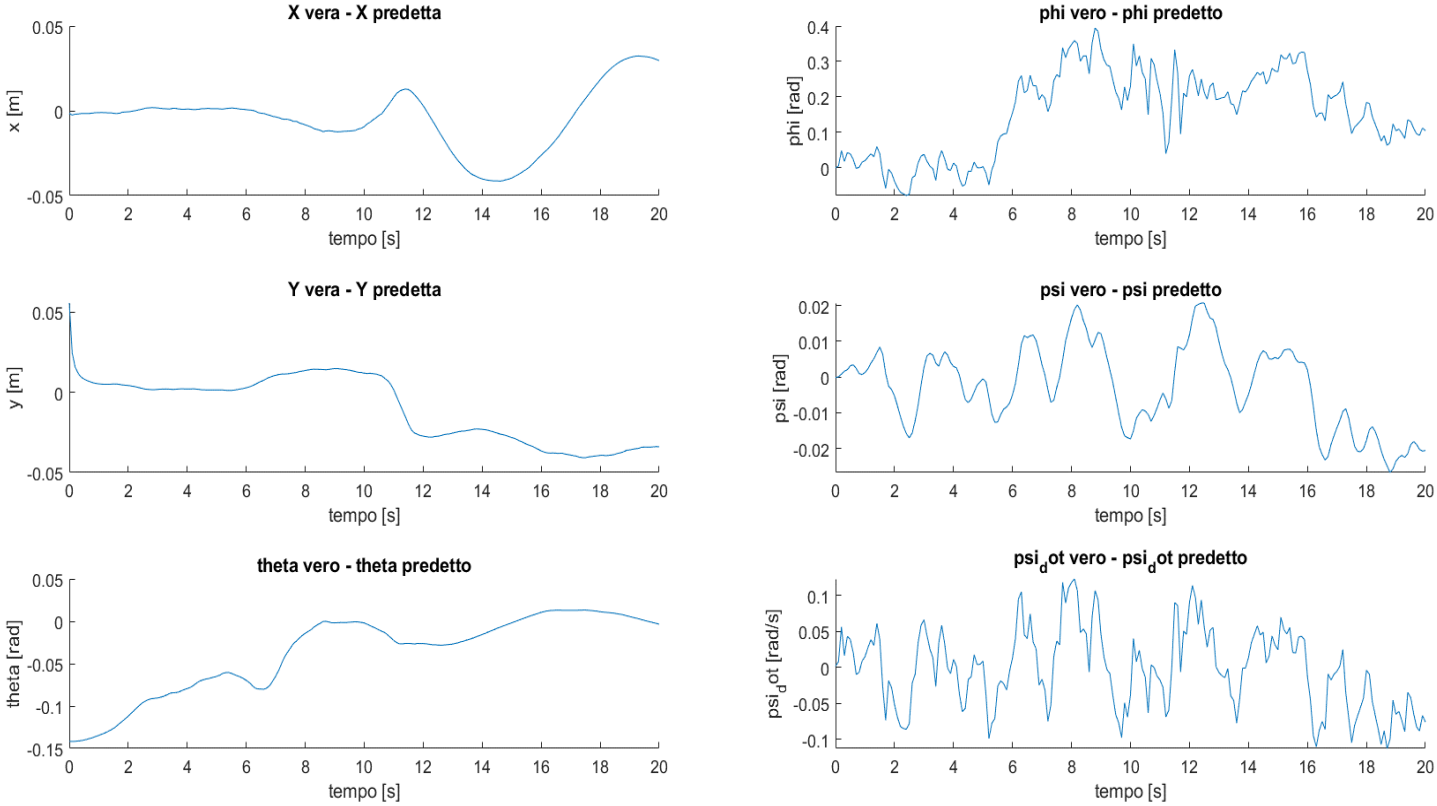


Per il filtro abbiamo i seguenti grafici:

- Grafico della norma dell'errore dato dalle coordinate X e Y vere e stimate
- Grafici dei valori veri e dei valori stimati delle variabili di stato
- Grafici dell'errore tra le variabili di stato vere e stimate







Come si può notare dal primo grafico la norma dell'errore all'inizio della simulazione è più grande per poi tendere a zero. Questo perché il filtro dopo un paio di secondi acquisisce sempre più informazioni che fanno assestare le variabili stimate molto vicino a quelle vere. Ciò si può anche notare dal secondo grafico dove le variabili vere e stimate sono molto vicine e l'errore nel terzo grafico è molto piccolo.

Questa simulazione è stata fatta con periodo di campionamento del filtro uguale a quella di tutti i sensori ed uguale a 0.1 sec. La prossima sfida affrontata è stata quella di imporre diversi periodi di campionamento dei sensori e del filtro.

2.2 EKF CON DIVERSI PERIODI DI CAMPIONAMENTO

Sono stati scelti i seguenti periodi di campionamento dei sensori:

t_sample_psi = 0.10012;

t_sample_phi_dot = 0.10010;

t_sample_dx = 0.10010;

t_sample_y_alpha = 0.1;

Tali valori entrano nei sensori nello schema Simulink ed in uscita è stata fatta la scelta di avere valori in formato ***Timeseries*** così si ha a disposizione il valore del sensore nel proprio istante di campionamento. Viceversa, nel caso precedente, i valori venivano passati a Matlab come ***double*** poiché i tempi di campionamento erano tutti uguali.

Nello script ***EKF2.mat*** è possibile trovare tale implementazione. I passaggi per lo sviluppo del filtro sono gli stessi nel caso dell'EKF visto in precedenza, in particolare la parte di predizione poiché essa non sfrutta le osservazioni. La parte che cambia è la correzione perché essa deve sfruttare le misure dei sensori e pertanto se la misura non arriva allora il filtro non deve correggere ed inoltre se la misura è stata già sfruttata per una precedente correzione, non deve essere riusata.

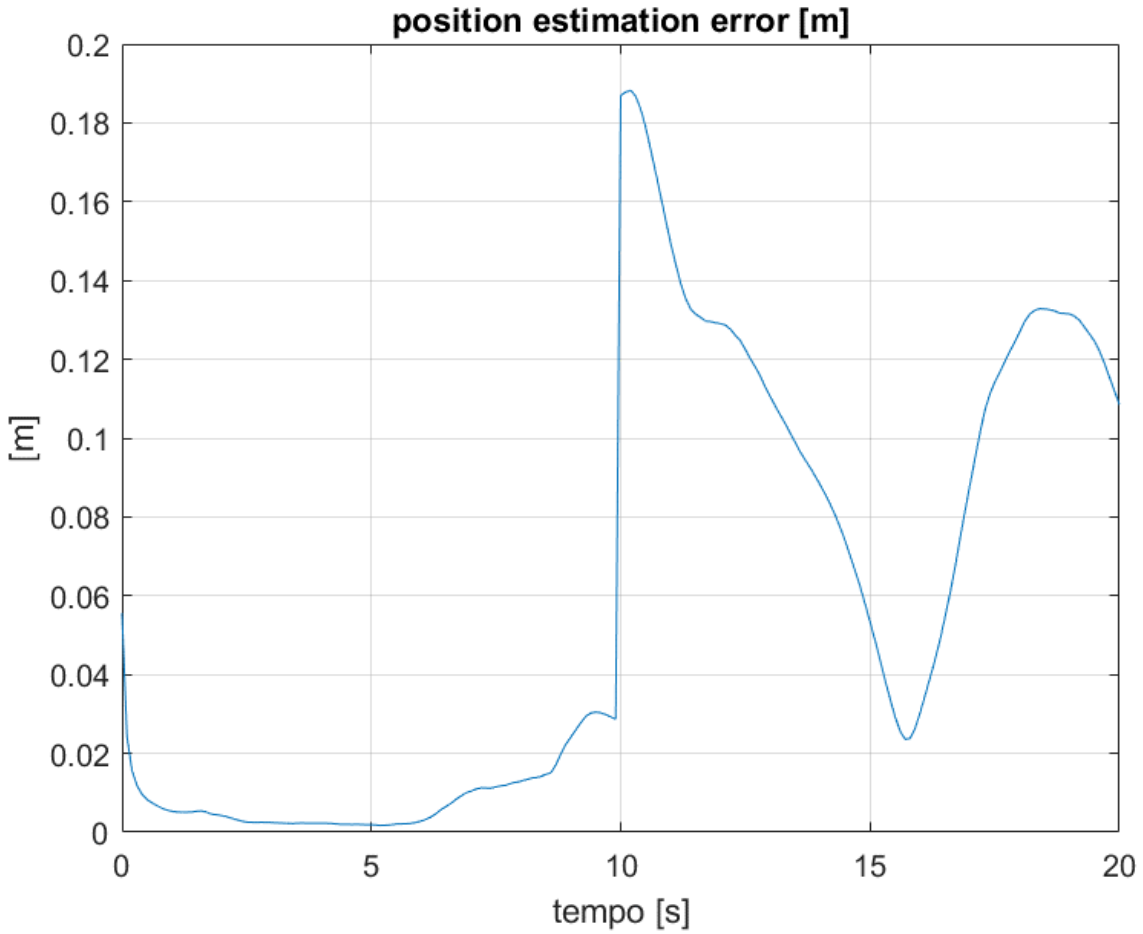
Per l'implementazione in Matlab si è scelto di sfruttare degli ***if*** nel for principale dell'EKF dove si va a confrontare il periodo di campionamento del filtro con il periodo di campionamento del sensore ad ogni passo, se il tempo del filtro è minore del tempo del sensore allora bisogna poi verificare che la misura al passo precedente è stata già usata. Per fare questo si è scelto di inserire in una struttura chiamata ***misure_utilizzate*** tutte gli istanti delle misure usate per la correzione così attraverso la funzione ***ismember*** si va a verificare che la misura al passo precedente non sia stata utilizzata quindi non faccia parte della struttura misure_utilizzate. A questo punto se la misura è stata utilizzata si mette un flag a 0 che verrà sfruttato nella funzione di innovazione per eliminare la riga o la colonna corrispondente al sensore non utilizzato delle matrici H,h,R,Mk. Questo perché se la misura non arriva il filtro non corregge con quella misura e pertanto il modello di osservazione perde di righe.

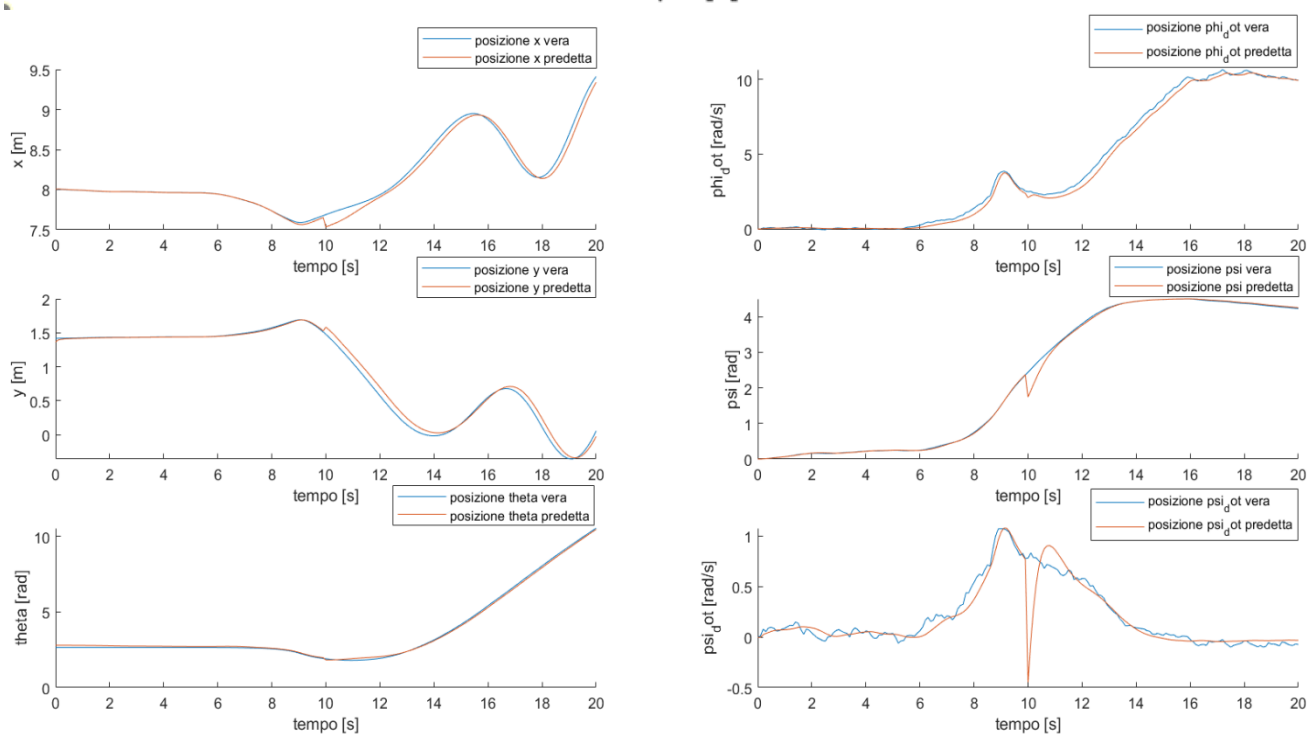
```
if T(k) < log_vars.psi_sens.Time(k)
    if ismember(misure_utilizzate.psi,log_vars.psi_sens.time(k-1)) ~= 0
     flag(k,1) = 1;
     misure_psi = log_vars.psi_sens.time(k-1);
else
    misure_psi = log_vars.psi_sens.time(k-1);
   log_vars.psi_sens.Data(k) = log_vars.psi_sens.Data(k-1);
    flag(k,1) = 0;
    end
end
```

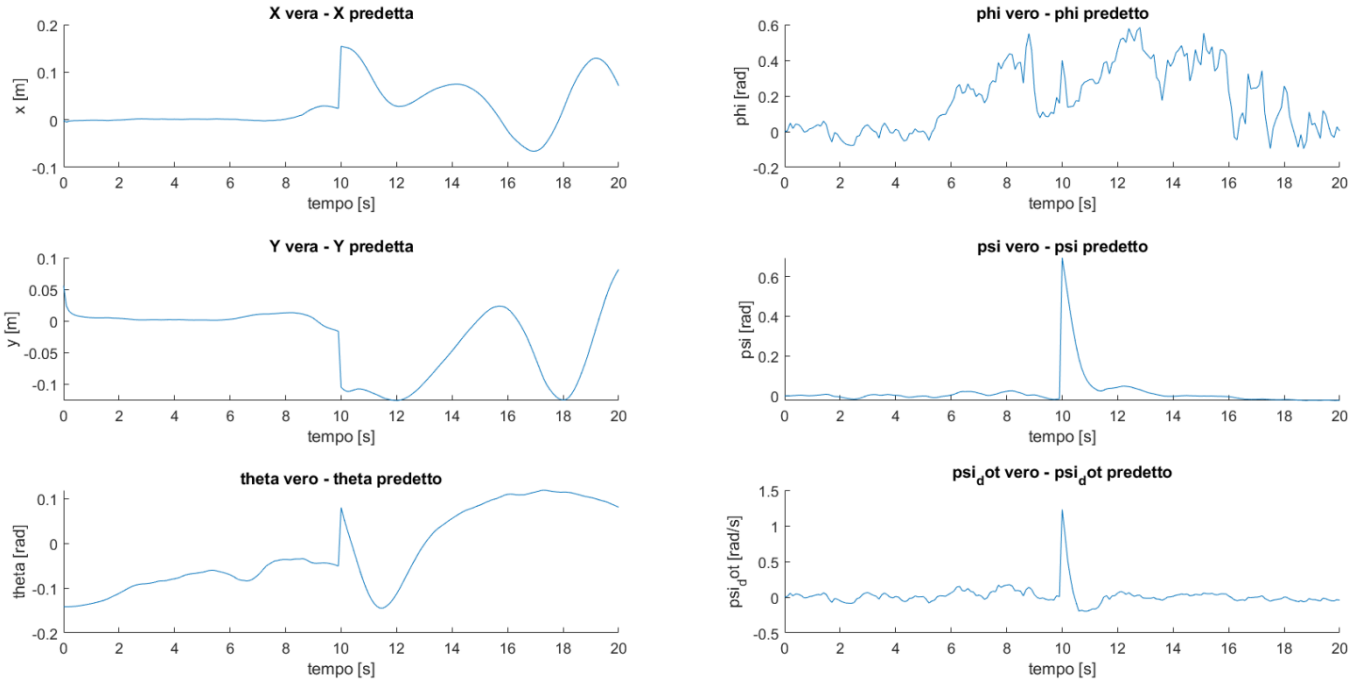
```
if flag(k,1) == 1
    H(1,:) = [];
    h(1) = [];
    R(1,:) = [];
    R(:,1) = [];
    Mk(1,:) = [];
    Mk(:,1) = [];
    misure(1) = [];
end
```

È stata fatta una simulazione con i seguenti ingressi ed ottenendo i seguenti grafici esplicati in precedenza.

- Grafico della norma dell'errore dato dalle coordinate X e Y vere e stimate
- Grafici dei valori veri e dei valori stimati delle variabili di stato
- Grafici dell'errore tra le variabili di stato vere e stimate







EKF CON SMOOTHER

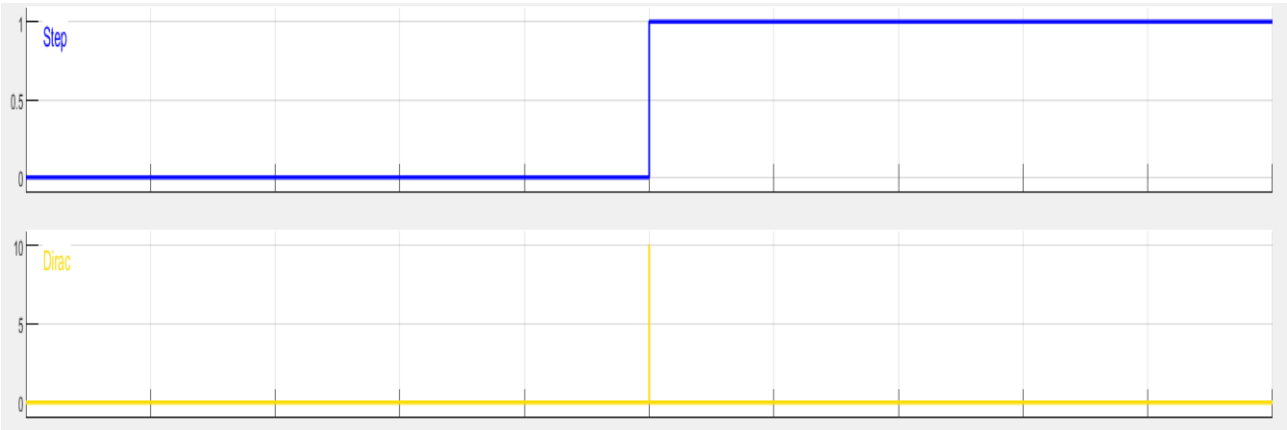
Un'altra sfida affrontata è stata quella del filtro di Kalman esteso con la regolarizzazione di Rauch,Tung,Strebel. La regolarizzazione è la stima all'istante k basandosi su misure fino all'istante n con n >=k . La procedura di regolarizzazione si compone di due fasi:

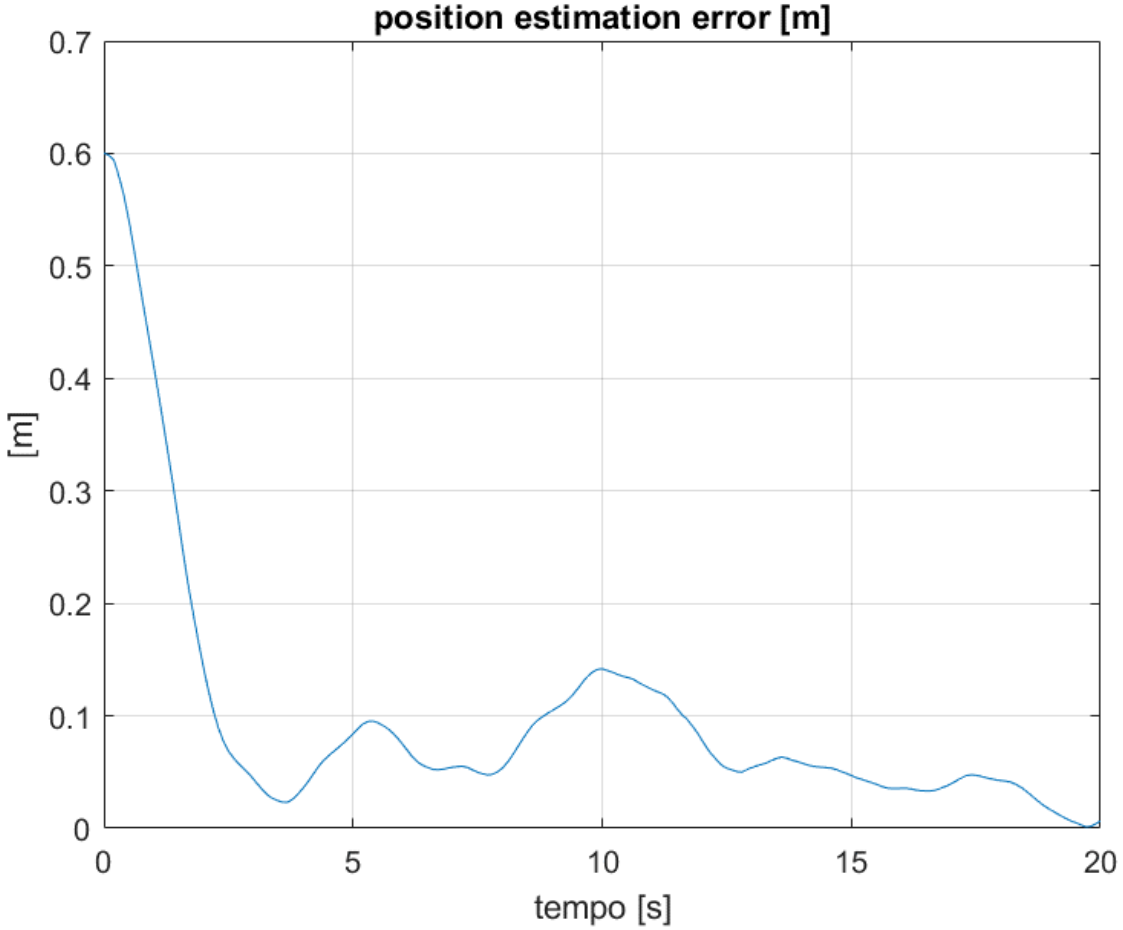
- AVANTI -> si implementa l'EKF o KF come visto dei paragrafi precedenti. La cosa che cambia a livello di codice nello script ***EKFwithsmoother.mat*** è che nel log_filter manteniamo anche la matrice F la quale ci servirà nella fase successiva.
- INDIETRO -> Dopo aver ottenuto tutte le x_hat e P dal filtro nella fase in avanti, esse serviranno per la fase a ritroso dove si sfrutteranno le formule conosciute a teorie. A questo punto si calcolano le nuove x_hat e P a ritroso.

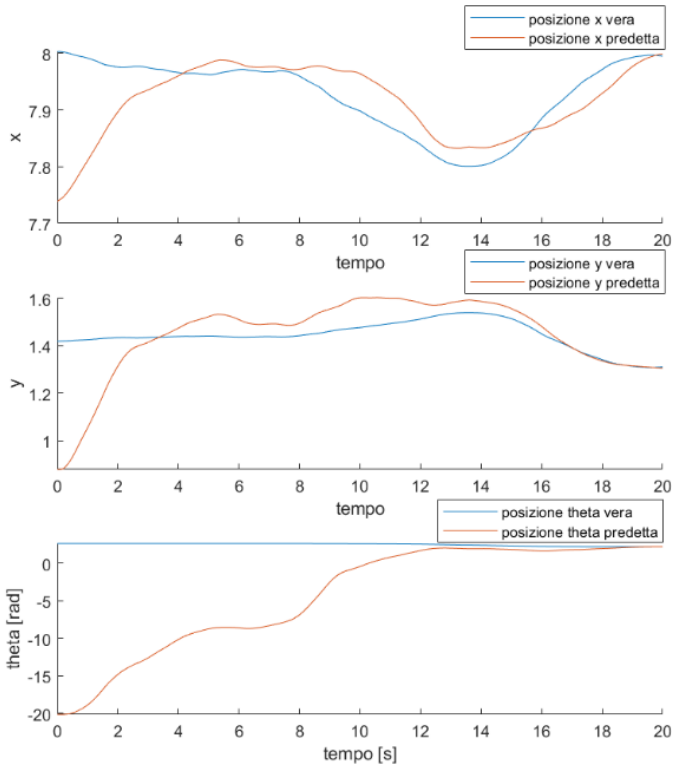
È possibile osservare la procedura di regolarizzazione nello script ***RtsSmoother.mat***. dove la fase a ritroso è mostrata nella seguente immagine. Si inizializza il codice in una struttura dove log_EKF dove ci inserisco due matrici x_hat_smoothed e P_smoothed dove le ultime colonne sono uguali alle x_hat e P in uscita dalla fase di correzione.

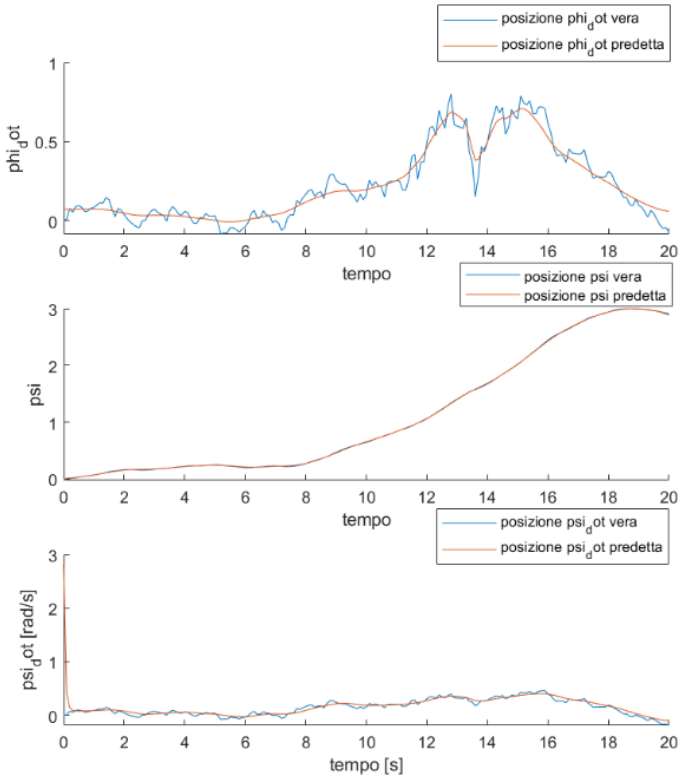
```
for k = size(log_EKF.x_hat_prediction,1)-1:-1:1 %k va da n-1 fino a 1 con step di 1

 % Ck = P_k|k * F_k+1' * P_k+1|k^(-1)
 Ck = [log_EKF.P_correction(k,:,1);log_EKF.P_correction(k,:,2);log_EKF.P_correction(k,:,3);log_EKF.P_correction(k,:,4);log_EKF.P_correction(k,:,5); ...
     log_EKF.P_correction(k,:,6)] ...
     * [log_EKF.F_matrix(k+1,:,1); log_EKF.F_matrix(k+1,:,2);log_EKF.F_matrix(k+1,:,3);log_EKF.F_matrix(k+1,:,4);log_EKF.F_matrix(k+1,:,5); ...
     log_EKF.F_matrix(k+1,:,6);]' / ...
     [log_EKF.P_prediction(k+1,:,1);log_EKF.P_prediction(k+1,:,2);log_EKF.P_prediction(k+1,:,3);log_EKF.P_prediction(k+1,:,4); ...
     log_EKF.P_prediction(k+1,:,5);log_EKF.P_prediction(k+1,:,6)];

 % x_k|n = x_k|k + Ck*(x_k+1|n - x_k+1|k)
 log_EKF.x_hat_smoothed(k,:) = log_EKF.x_hat_correction(k,:)' + Ck*(log_EKF.x_hat_smoothed(k+1,:)' - log_EKF.x_hat_prediction(k+1,:)');


  % P_k|n = P_k|k + Ck*(P_k+1|n - P_k+1|k)*Ck'
 log_EKF.P_smoothed(k,:,:) = [log_EKF.P_correction(k,:,1);log_EKF.P_correction(k,:,2);log_EKF.P_correction(k,:,3); ...
     log_EKF.P_correction(k,:,4);log_EKF.P_correction(k,:,5);log_EKF.P_correction(k,:,6)] ...
      + Ck*([log_EKF.P_smoothed(k+1,:,1);log_EKF.P_smoothed(k+1,:,2);log_EKF.P_smoothed(k+1,:,3); ...
      log_EKF.P_smoothed(k+1,:,4);log_EKF.P_smoothed(k+1,:,5);log_EKF.P_smoothed(k+1,:,6)] - [log_EKF.P_prediction(k+1,:,1); ...
      log_EKF.P_prediction(k+1,:,2);log_EKF.P_prediction(k+1,:,3);log_EKF.P_prediction(k+1,:,4);log_EKF.P_prediction(k+1,:,5); ...
      log_EKF.P_prediction(k+1,:,6)])*Ck';
end
```

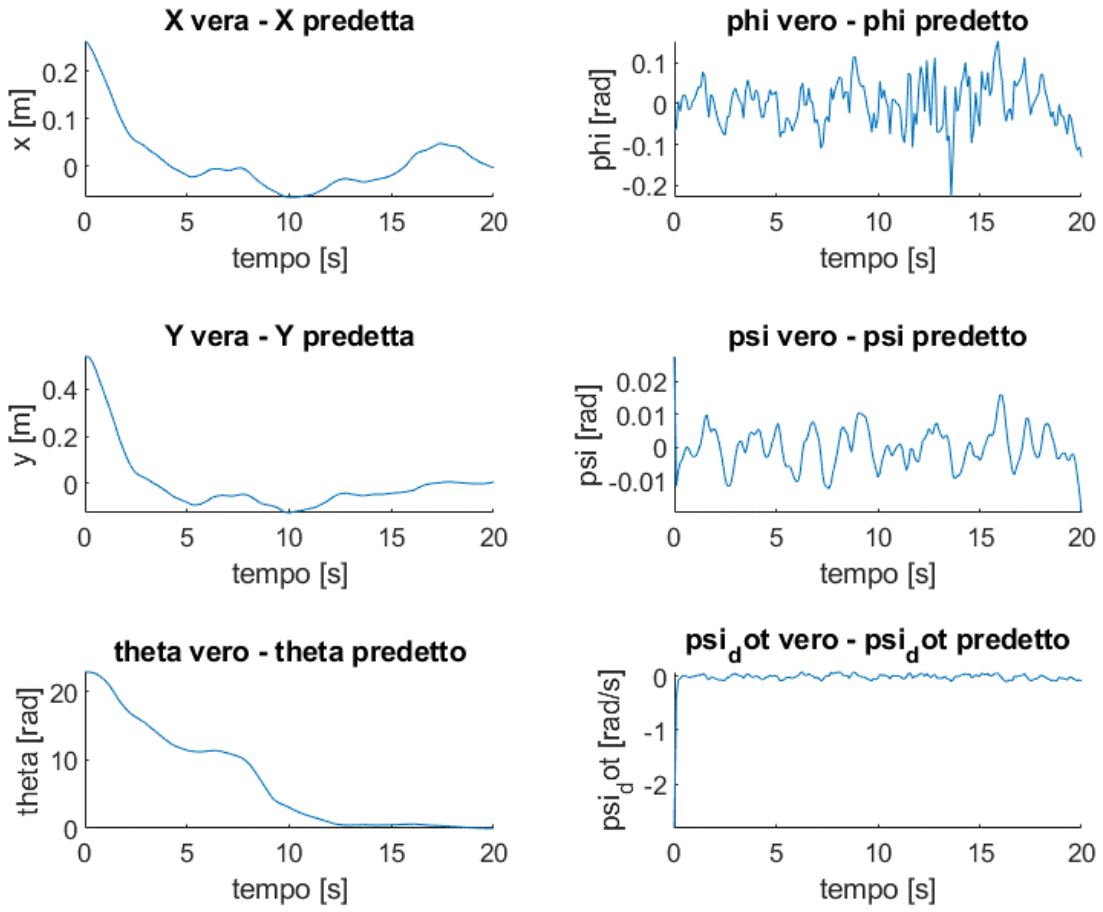
È stata fatta una simulazione per mostrare il comportarsi del filtro con lo Smoother, sfruttando gli stessi dati e segnali di ingresso dell'EKF.




position estimation error [m]
[m]
tempo [s]


posizione x vera
posizione x predetta
x
tempo
posizione y vera
posizione y predetta
y
tempo
posizione theta vera
posizione theta predetta
theta [rad]
tempo [s]


posizione phi_dot vera
posizione phi_dot predetta
phi_dot
tempo
posizione psi vera
posizione psi predetta
psi
tempo
posizione psi_dot vera
posizione psi_dot predetta
psi_dot [rad/s]
tempo [s]



Come si può notare dopo una prima fase nel quale l'errore di stima è elevato, la stima si assesta vicino al valore vero con errore molto piccolo. Inoltre si può notare che l'andamento delle variabili regolarizzate è più regolare, si hanno meno salti.

2.3 FILTRO UKF

Il filtro di Kalman Unscented è un tipo di filtro che sfrutta la trasformata Unscented. Tale trasformata è un metodo di approssimazione dei momenti di una variabile aleatoria ottenuta mediante una trasformazione non lineare di un'altra variabile aleatoria di cui conosciamo una caratterizzazione statistica, in particolare il suo valor medio e la varianza. Quello che si ottiene da questa trasformata è il valor medio, la varianza e la matrice di covarianza tra la variabile aleatoria iniziale e quella ottenuta mediante trasformazione. Le grandezze in input alla trasformata sono: la media e la varianza della v.a che si conosce e il modello non lineare. Queste grandezze ci servono per effettuare una stima BLUE (best linear unbiased estimator) cioè trovare lo stimatore ottimo lineare MMSE

(minimum mean square error) che è uno stimatore che minimizza l'errore quadratico medio di stima. Il filtro di Kalman Unscented serve per problemi di filtraggio di tipo non lineari come il caso in esame. Essa si compone sempre di due parti:

- PREDIZIONE : la trasformata serve per ottenere x^k+1/k avendo a disposizione x^k/k. Si sfrutta il modello del sistema non lineare con l'obiettivo di ricavare i momenti di x_k+1 noti i momenti di x_k. I 3 input della trasformata sono: ([x^k/k ; 0], [P_k/k 0 ; 0 Q_k], f( )) ottenendo x^k+1/k , Pk+1/k e il momento incrociato che non serve.
- CORREZIONE : la trasformata serve per ottenere x^k/k avendo a disposizione x^k/k-1. Si sfrutta il modello di osservazione non lineare. I 3 input della trasformata sono: ([x^k/k-1 ; 0], [P_k/k-1 0 ; 0 R_k], f( )) ottenendo y^k/k-1 ,la matrice di covarianza associata S_k e P_k/k-1 (di x e y). Tutti e tre i momenti servono per la stima BLUE ottenendo cosi il guadagno di correzione L_k , x^k/k e P_k/k dalle formule viste a teoria.

Finora è stata spiegata la teoria che c'è dietro la trasformata, ora essa sarà applicata al caso di studio in esame ed in particolare all'implementazione in Matlab. È possibile trovare ciò nello script ***UKF_filter.mat.***

Come si è già visto per l'EKF, si hanno una serie di variabili in uscita da Simulink che vengono messe tutte in una struttura chiamata log_vars. Subito dopo c'è il corpo principale del filtro con il ciclo for che contiene la parte di predizione e la parte di correzione. La funzione di predizione contiene esclusivamente un'altra funzione dedicata alla trasformata Unscented. In input a tale funzione si possono notare le grandezze viste precedentemente dalla teoria, in particolare un blocco che contiene le medie (ci sono due zeri poiché le medie dei rumori di processo sono nulli) ed un blocco diagonale che contiene la matrice P del passo precedente e la matrice Q. Stessa cosa per la funzione di correzione che contiene a sua volta la funzione di trasformata Unscented. Diversamente dalla predizione la correzione sfrutta il modello di osservazione ed essendo il rumore additivo, non si considera negli input della trasformata. Nella funzione di correzione si trovano ulteriori formule viste a teoria per la stima BLUE.

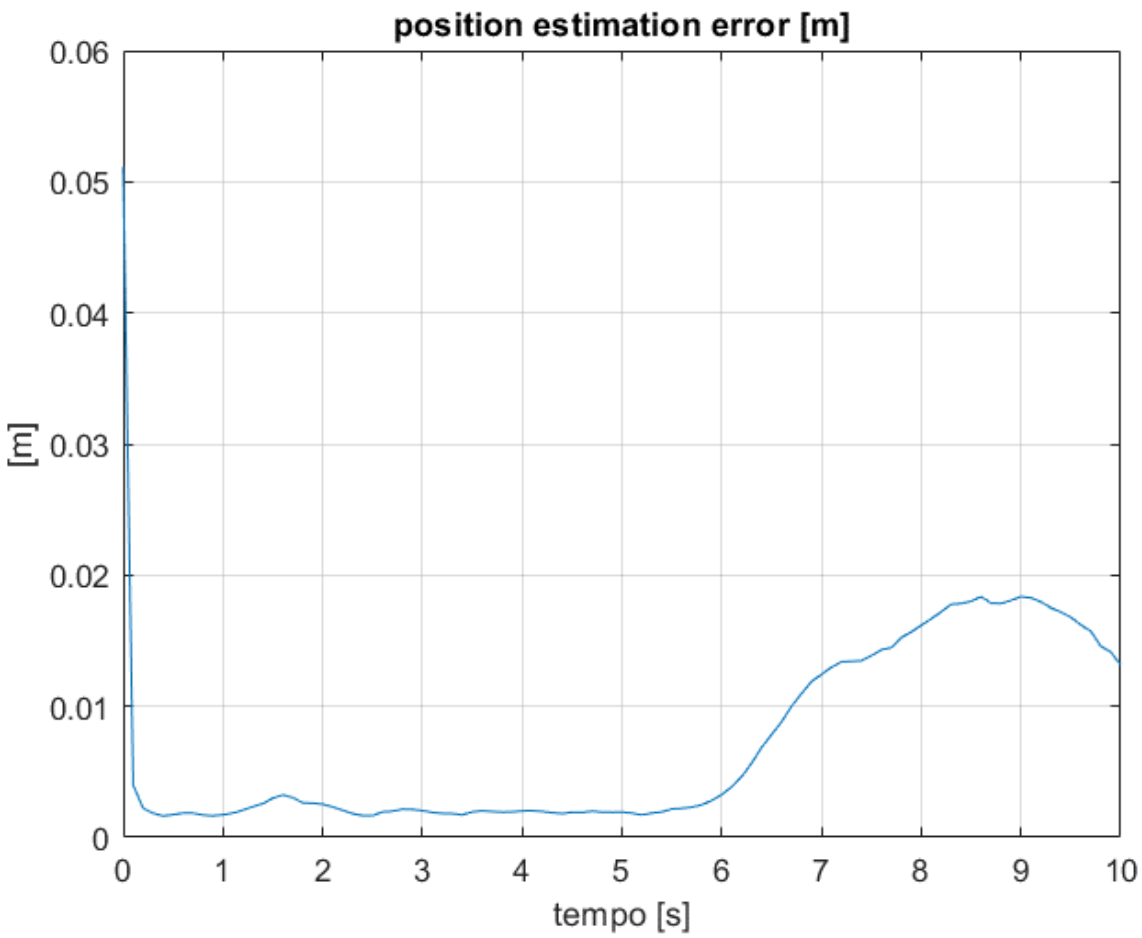
Subito dopo è possibile trovare un'immagine con entrambe e funzioni spiegate sopra.

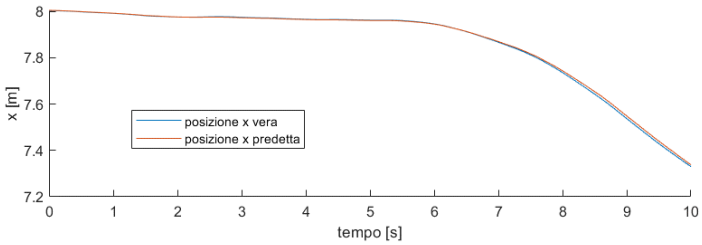
```
function [x_hat,P] = predict_UKF(x_hat,P,Q,dt,tau_phi,tau_psi,ht)
   [x_hat,P] = UnscentedTransform_F_Function([x_hat; 0; 0], blkdiag(P, Q), dt, tau_phi, tau_psi,ht);
end

function [x_hat,P,e] = correct_UKF(x_hat, P, R,dx_sens,phi_dot_sens,psi_sens,y_alpha,ht1)
 [hat_y, S, Pxy] = UnscentedTransform_H_Function(x_hat, P,ht1);
   y = [psi_sens; phi_dot_sens; dx_sens ;y_alpha];
   S = S + R;
   e = y - hat_y;
  e(1) = atan2(sin(e(1)),cos(e(2)));
  e(4) = atan2(sin(e(4)),cos(e(4))); %la differenza di angoli deve essere compresa tra -pi e pi
   L = Pxy*S^(-1);

   x_hat = x_hat + L*e;
   P = P - L*S*L';
end
```

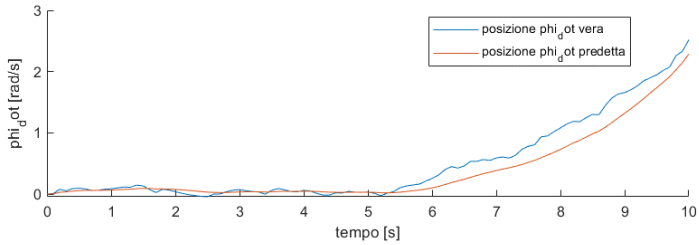
Per quanto riguarda le funzioni dedicate alla trasformata Unscented, sono state costruite in modo meccanico conoscendo la teoria. I passi sono i seguenti:

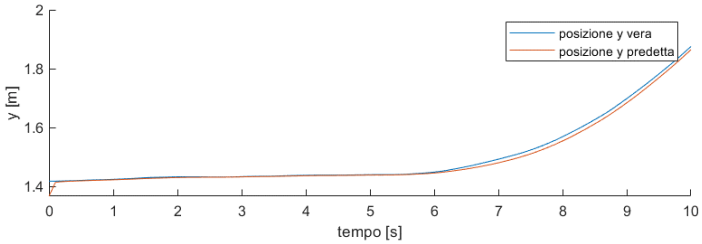
- Inizializzazione parametri alpha,beta,kappa. Nel caso in esame alpha = 1 , beta = 2 , k = 0. Questa è la scelta ottimale nel caso gaussiano inoltre con alpha = 1 ,lambda = kappa.
- Calcolo dei pesi
- Fattorizzazione matrice delle covarianze sfruttando la decomposizione ai valori singolari. Essa sarà scritta come gamma * gamma trasposto.
- Generazione degli 2n + 1 sigma points. Utile nel caso pratico poiché simmetria tra i sigma points. Il sigma point centrale coincide con la media.
- Dopo aver calcolato i sigma points (x_i), a questi bisogna applicare la funzione f del modello non lineare del sistema, in modo tale da ottenere i sigma points propagati. Nel caso della trasformata Unscented nella correzione i sigma points devono essere propagati nel modello di osservazione.
- Calcolo delle approssimazioni dei momenti come medie campionarie. La media pesata tra angoli è calcolata facendo l'atan2(sin(sigma point propagato * peso),cos(sigma point propagato * peso)). Poi si è anche calcolato il tilde propagato che è la differenza rispetto alla media che serve per il calcolo della matrice di covarianza campionaria. La differenza principale che c'è tra la trasformata Unscented della predizione e correzione è che per la predizione non serve il momento incrociato e quindi non calcolato invece nella correzione serve.

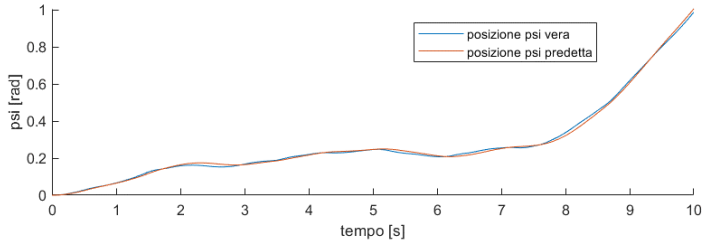
Per dimostrare che il filtro UKF funzioni correttamente è stata fatta una simulazione con gli stessi parametri di ingresso della simulazione sull'EKF. I risultati sono mostrati nelle figure seguenti.

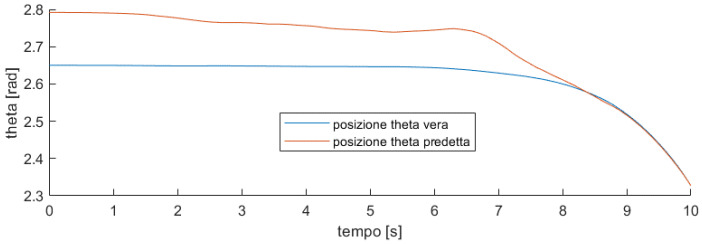


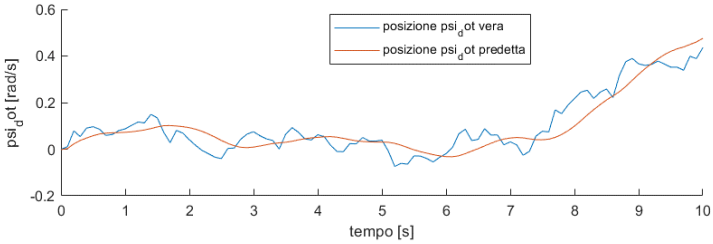

posizione x vera
posizione x predetta
x [m]
tempo [s]

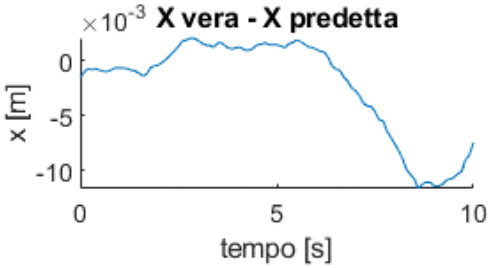

posizione phi_dot vera
posizione phi_dot predetta
phi_dot [rad/s]
tempo [s]


posizione y vera
posizione y predetta
y [m]
tempo [s]

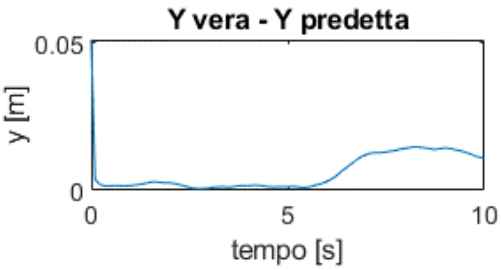

posizione psi vera
posizione psi predetta
psi [rad]
tempo [s]

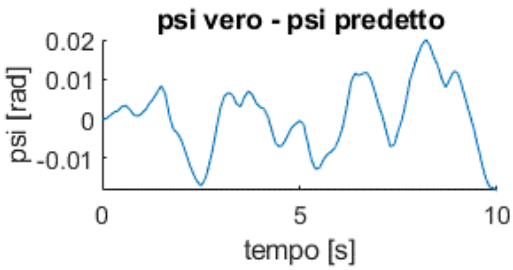

posizione theta vera
posizione theta predetta
theta [rad]
tempo [s]

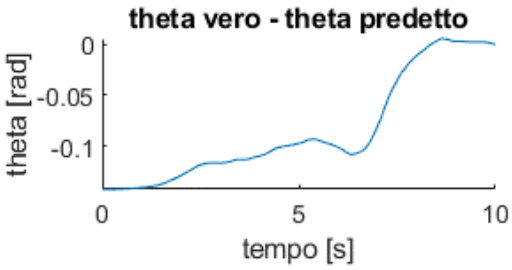

posizione psi_dot vera
posizione psi_dot predetta
psi_dot [rad/s]
tempo [s]

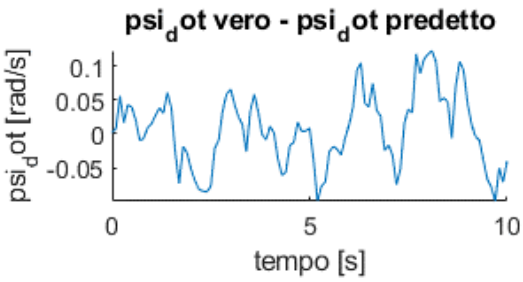

X vera - X predetta
x [m]
tempo [s]

phi vero - phi predetto
phi [rad]
tempo [s]


Y vera - Y predetta
y [m]
tempo [s]

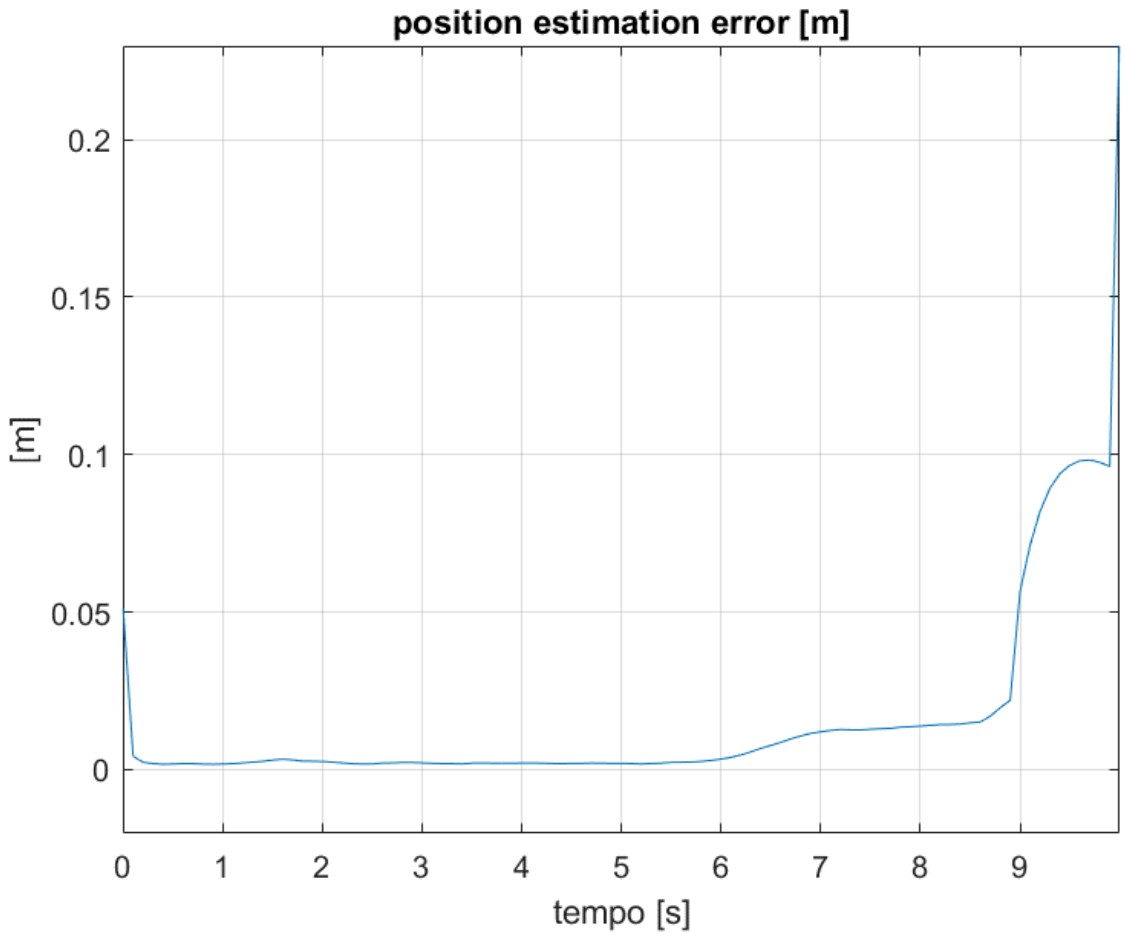

psi vero - psi predetto
psi [rad]
tempo [s]

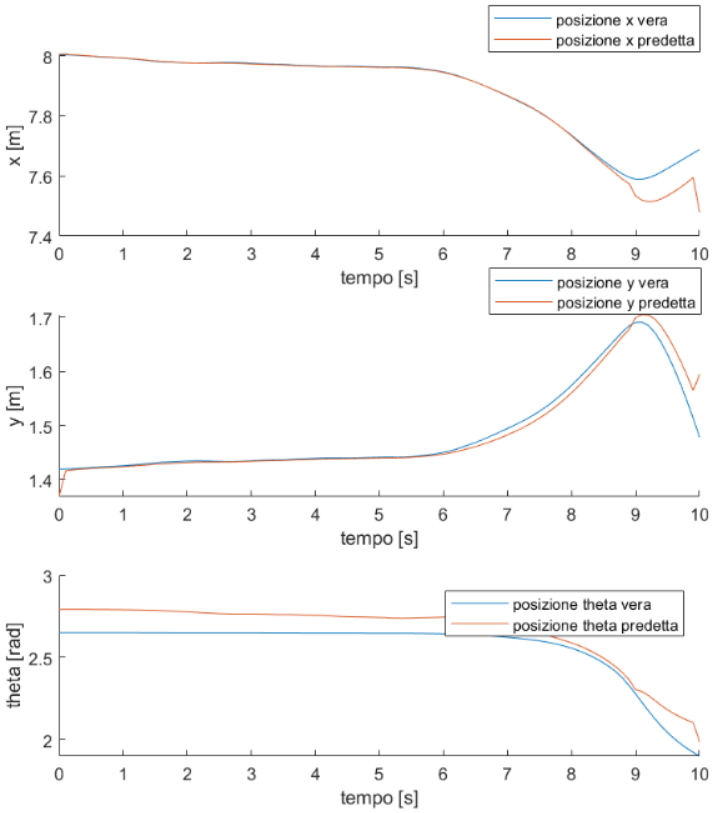

theta vero - theta predetto
theta [rad]
tempo [s]

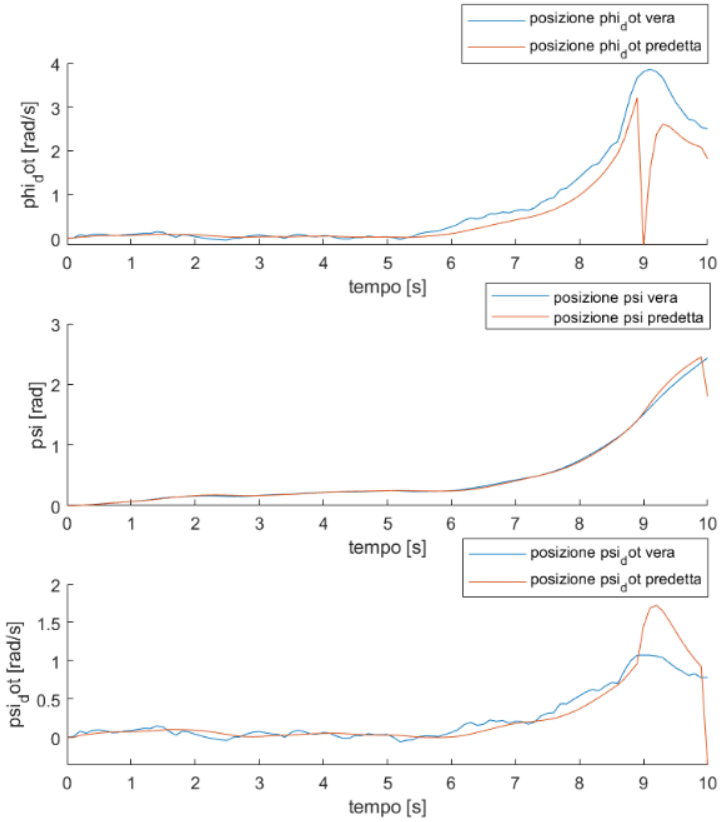

psi_dot vero - psi_dot predetto
psi_dot [rad/s]
tempo [s]

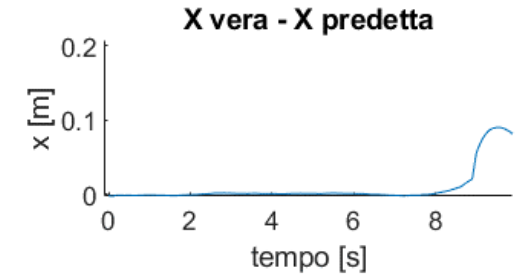
Come si può notare dal primo grafico la norma dell'errore all'inizio della simulazione è più grande per poi tendere a zero. Questo perché il filtro dopo un paio di secondi acquisisce sempre più informazioni che fanno assestare le variabili stimate molto vicino a quelle vere. Ciò si può anche notare dal secondo grafico dove le variabili vere e stimate sono molto vicine e l'errore nel terzo grafico è molto piccolo.

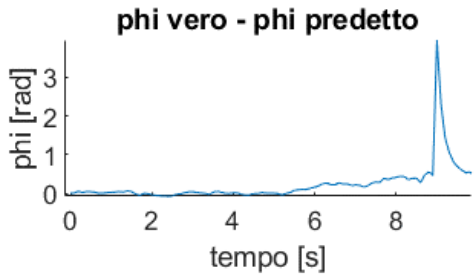
Questa simulazione è stata fatta con periodo di campionamento del filtro uguale a quella di tutti i sensori ed uguale a 0.1 sec. Come visto per l'EKF, è stata implementata anche per l'UKF una versione in cui il periodo di campionamento del filtro è diverso dal periodo di campionamento dei sensori. L'implementazione segue la stessa logica vista per l'EKF l'unica differenza sostanziale si ha nella parte di correzione dove le matrici R,S e Pxy in uscita dalla trasformata Unscented perdono righe e colonne in base alla misura che non si usa per correggere, come visto per l'EKF. Anche in questo caso è stata fatta una simulazione dove le seguenti immagini mostrano i risultati.

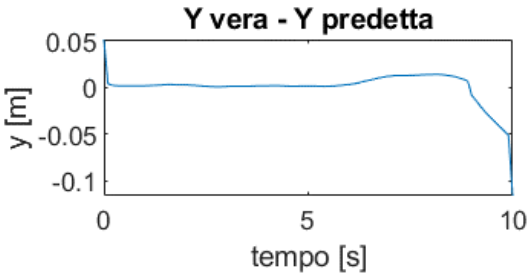


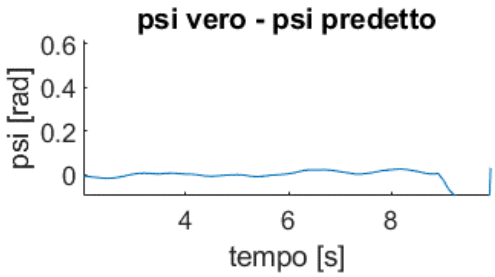

posizione x vera
posizione x predetta
x [m]
tempo [s]
posizione y vera
posizione y predetta
y [m]
tempo [s]
posizione theta vera
posizione theta predetta
theta [rad]
tempo [s]

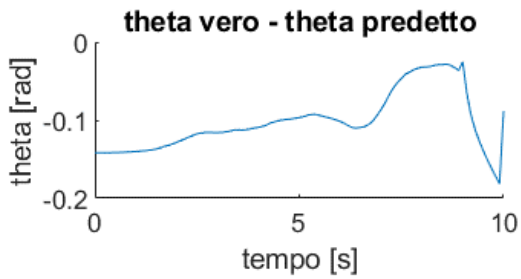

posizione phi_dot vera
posizione phi_dot predetta
phi_dot [rad/s]
tempo [s]
posizione psi vera
posizione psi predetta
psi [rad]
tempo [s]
posizione psi_dot vera
posizione psi_dot predetta
psi_dot [rad/s]
tempo [s]

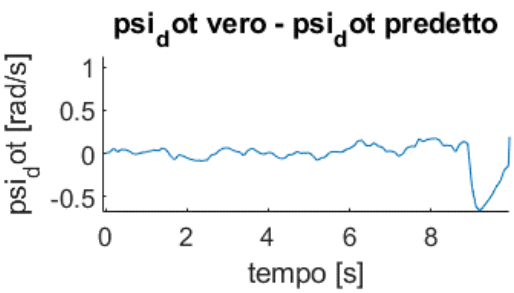

X vera - X predetta
x [m]
tempo [s]


phi vero - phi predetto
phi [rad]
tempo [s]


Y vera - Y predetta
y [m]
tempo [s]


psi vero - psi predetto
psi [rad]
tempo [s]


theta vero - theta predetto
theta [rad]
tempo [s]


psi_dot vero - psi_dot predetto
psi_dot [rad/s]
tempo [s]

Conclusioni

Per analizzare i risultati ottenuti dai filtri è stato necessario andare a confrontare con ingressi diversi e quindi con valori delle coppie e angoli di sterzo diversi tra loro, le varie traiettorie vere e stimate. Per completezza si è anche analizzato il caso in cui le condizioni iniziali siano diverse da quelle di partenza.

Come visto in teoria per misurare la bontà del filtraggio e dimostrare che il filtro risponde bene anche nelle varie condizioni di incertezza, deve verificarsi che si arrivi ad una convergenza delle variabili di stato stimate con quelle vere.

EFK -> Nello studio del progetto, si può dimostrare che il filtro stima bene la traiettoria del veicolo, coerentemente con i rumori applicati in ingresso e in uscita. Inoltre si otterranno risultati interessanti anche ipotizzando di partire da condizioni iniziali diverse.

Si è verificato che anche con condizioni iniziali distanti da quelle reali il filtro EFK converge alla traiettoria vera abbastanza velocemente e in poche iterazioni.

In generale andando ad aumentare il disco di incertezza aumentando la varianza sulle misurazioni, riusciamo ad ottenere ancora delle buone stime con traiettorie stimate molto vicine a quelle reali, con risultati pressoché simili a quelli ottenuti con incertezze più piccole.

Valutando l'errore, in tutti i grafici si potrà notare che, salvo un piccolo transitorio iniziale, su ogni variabile di stato l'errore sarà prossimo allo zero.

Ulteriori miglioramenti ci aspettiamo possano essere fatti, andando per esempio a considerare un modello dinamico leggermente più complesso che tenga conto di grandezze fisiche che si sono trascurate nel nostro modello semplificato.

Codice Matlab

INIT.MAT

```
 clear all

close all
clc

x_start = [0 0 0]';

x_meas = [0 0 0]';

L = 1.5; %m
rp = 0.2; %m
l=0.8; %m
IPy=0.045;
IPz=3.5;
Mv=80; %kg
mp=2; %kg
ma=2; %kg
IAy=0.025;
ra=0.20; %m
d=0.4; %m
IGz=7.5;
IAz=7;
a = (Mv + mp + 2*ma + (2*IAy/(ra^2)));
b = 1/2*(Mv*(l^2) + mp*(L^2) + 2*ma*(d^2) +IGz + IPz + 2*IAz + 2*IAy*((d/ra)^2));
c = Mv*l + mp*L;


t_sample = 0.1;
sensor_var = 0;

%Costruzione modello su Matlab

%Vettore di stato -> x1 = x ; x2 = y ; x3 = theta ; x4 = phi_dot ; x5 = psi
% x6 = psi_dot

% tau = [tau_phi tau_psi] %vettore degli ingressi
tau= sym('tau',[ 2 1 ],'real');

% Vettore di stato in forma simbolica

X = sym('X',[6 1 ],'real');


%Inizializzazione variabili ( non conoscendo la posizione iniziale la
%ipotizzo random

limits = 10;

x_0 = limits * rand(1); % posizione iniziale x

y_0 = limits * rand(1); %posizione iniziale di y
```

```
theta_0 = 2*pi*rand(1); %imposto theta angolo che varia tra 0 e 360 gradi dovremmo
inserire 0,1 intervallo

phi_dot_0 = 0; %suppondendo che la velocità iniziale della ruota posteriore è nulla

psi_dot_0 = 0; %supponendo che la velocità iniziale sterzante nulla

psi_0 = 2*pi*rand(1); %supponogo random la posizione dello sterzo iniziale anche qui
0,1


%Non sappiamo dove si trova l'AGV ma stiamo ipotizzando che si trovi in un
%campo 10mx10m con disco di incertezza 0.3 inoltre suppongo incerto anche
%l'angolo di sterzo invece le velocità iniziali nulle

std_dev_init_x_y = 0.5 ;%supponendo disco di incertezza 0.5
std_dev_init_angoli = 0.1 ; %supponendo varianza di 0.6 gradi
std_dev_init_vel_ang = 0.05 ; %supponendo varianza di 0.15 rad/s
xr_0 = x_0 + std_dev_init_x_y*randn(1); %posizione x iniziale sporcata
yr_0 = y_0 + std_dev_init_x_y*randn(1); %posizione y iniziale sporcata
thetar_0 = theta_0 + std_dev_init_angoli* randn(1); %posizione theta iniziale sporcata
psir_0 = psi_0 + std_dev_init_angoli*randn(1); %posizione psi iniziale sporcata

x_hat = [xr_0 yr_0 thetar_0 0 psir_0 0]';
P = blkdiag(std_dev_init_x_y , std_dev_init_x_y, std_dev_init_angoli ,
std_dev_init_vel_ang ,std_dev_init_angoli ,std_dev_init_vel_ang)^2; %suppongo che la
matrice delle covarianze abbia queste varianze(che posso modificare) il che significa
%che le misure si discostano di 0.3 dal valore vero.


        %Caratteristiche sensori
V = sym('v',[4 1],'real'); %rumore aggiunto ai sensori ci sarà utile nella h

dev_psi = 0.0017; %circa 0,1 gradi.
        % La precisione dell'encoder è una misura dell'errore tra il valore letto
dall'encoder
        % e il valore fisico effettivo misurato.
        % La precisione dell'encoder è misurata in minuti d'arco o secondi d'arco con
20 minuti d'arco (0,3 gradi)
        % o meglio generalmente considerato un encoder ad alta precisione
        % con alcuni dispositivi di precisione dell'ordine di 5 secondi d'arco (0,0014
gradi).
dev_phi_dot = 0.01; %rad/s

dev_dx = 0.02; %m
dev_yalpha= 0.0017; %rad

%Anche gli ingressi possono essere affetti da rumore come ipotizzato anche nel modello
simulink
W = sym('w',[2 1],'real'); %w indica il rumore in ingresso all'ingresso ci sarà utile
nella f

dev_w1 = 0.04;
dev_w2 = 0.04;

%Calcolo di h. Come sappiamo l'h si riferisce al modello di osservazione
%quindi al modello del sensore. In questo caso è molto semplice calcolarlo
%poichè dobbiamo solo considerare le 4 uscite dei sensori che esprimiamo
%dalle variabili di stato ed inoltre aggiungiamo rumore poichè sappiamo che
%alle misure aggiungiamo rumore;
```

```
y_alpha = atan2(X(2),X(1));
h = [X(5) + V(1) , X(4) + V(2) , X(1) + V(3) ,  y_alpha + V(4) ];

% con x1 = x ; x2 = y ; x3 = theta ; x4 = phi_dot ; x5 = psi
% x6 = psi_dot


%Ora definiamo la f (modello del sistema) la quale ci sarà utile nell'implementazione
del filtro
%di Kalman

%La f è associata al modello del sistema e quindi all'equazioni della
%dinamica e della cinematica

%Quello che bisogna fare per scrivere f è per prima cosa come fatto nello
%schema simulink scrivere tutto nella forma B*q_ddot + C(q,q_dot) = tau
%sostituendo le variabili di stato costruite come variabili simpoliche
%(utile per caloclare jacobiano dopo). Nella f inoltre dobbiamo anche tener
%conto del rumore del segnale di ingresso ecco perchè alla tao sommiamo W
%che è il rumore( tutte variabili simboliche poichè dopo costruiremo le
%matrici a partire da queste con le derivate).(rendendo così tau=tau_w)
% Come sappiamo per il calcolo della f bisogna calcolare x_dot (vettore di stato),
discretizzare
% e formula X_k+1 = X_k + dt* X_dot con X_k condizione iniziale

B = [a*rp^2*cos(X(5))^2+b*(rp/L)^2*sin(X(5))^2+IPy, -IPz*(rp/L)*sin(X(5)); -
IPz*(rp/L)*sin(X(5)), IPz];


C = [(b*(rp/L)^2-a*rp^2)*X(4)*X(6)*sin(X(5))*cos(X(5)); -
IPz*(rp/L)*X(4)*X(6)*cos(X(5))];
tau_w = [tau(1)+ W(1) tau(2)+W(2)] ;

M = (inv(B)*((tau_w)' - C ));

%Calcolo X_dot

x1_dot=rp*X(4)*cos(X(3))*cos(X(5));
x2_dot=rp*X(4)*sin(X(3))*cos(X(5));
x3_dot=-(rp*X(4)*sin(X(5)))/L;
x4_dot=M(1); %phi_ddot
x5_dot=X(6); %psi_dot
x6_dot=M(2); %psi_ddot

X_dot=[x1_dot x2_dot x3_dot x4_dot x5_dot x6_dot]';

%Ora si passa a implementare il filtro di Kalman esteso con annesse matrici
%F Q R D H M, Sappiamo dalla teoria che il filtro di Kalman esteso è per
%sistemi non lineari e pertanto si dovrà effettuare una linearizzazion al
%primo termine per poi discretizzare e derivare.


%Si sono definite le 6 matrici d'interesse R Q F H D M, in particolare, F D tramite lo
Jacobiano della
%funzione f definita :f=X+dt*X_dot come già prima enunciato f è associata al modello
del sistema
% e quindi all'equazioni della dinamica e della cinematica e H M tramite lo
% jacobiano della funzione h (si riferisce al modello di osservazione
%quindi al modello del sensore sporcato dal rumore additivo.
```

```
dt=t_sample;% tempo di campionamento
f=X+dt*X_dot;

R=blkdiag(dev_psi^2,dev_phi_dot^2,dev_dx^2,dev_yalpha^2); %R è la matrice in cui sono
presenti sulla diagonale le varianze (deviazioni al quadrato) dei sensori

Q=blkdiag(dev_w1^2,dev_w2^2); %Q è la matrice in cui sono presenti sulla diagonale le
varianze (deviazioni al quadrato) degli ingressi

F=jacobian(f,X); %la F si calcola derivando la f per gli stati quindi sarà una matrice
6x6

D=jacobian(f,W); % la D si calcola derivando la f per il rumore applicato applicato
agli ingressi quindi sarà una matrice 6x2 ed inoltre essendo il
%rumore presente sugli ingressi quindi le uniche due righe diverse da zero
%saranno la 4 e la 6

H=jacobian(h,X); %la H si calcola derivando la h per gli stati quindi sarà una matrice
4x6

Mk=jacobian(h,V); % la M si calcola derivando la h per gli ingressi rumorosi, siccome
gli ingressi rumorosi sono additivi allora la matrice sarà identità 4x4


%Adesso implemento il filtro con i diversi periodi di campionamento dei sensori
t_sample_psi = 0.10012;
t_sample_phi_dot = 0.10010;
t_sample_dx = 0.10010;
t_sample_y_alpha = 0.1;
```

EKF.MAT

```
%Abbiamo i dati che fuoriescono dai sensori che noi preendiamo da Simulink.
%Questi dati li prendiamo e li mettiamo in un'unica struttura che ci
%servirà per inizializzare l'EKF


t_simulation=10;
out=sim('isiprojectsimulink',t_simulation);
log_vars = [];
log_vars.X = (out.X)';
log_vars.Y = (out.Y)';
log_vars.dx_sens = (out.dx_sens)';
log_vars.phi_dot = (out.phi_dot)';
log_vars.phi_dot_sens = (out.phi_dot_sens)';
log_vars.psi = (out.psi)';
log_vars.psi_dot= (out.psi_dot)';
log_vars.psi_sens = (out.psi_sens)';
log_vars.tau_phi = (out.tau_phi)';
log_vars.tau_psi = (out.tau_psi)';
log_vars.theta = (out.theta)';
log_vars.y_alpha = (out.y_alpha)';
log_vars.x_hat = x_hat;
log_vars.P = P;
```

```
k=0;
 for p=0:dt:t_simulation
     k=k+1;
 log_vars.t(k)=p;
end

 log_vars.dt = t_sample;
 log_vars.t_simulation = t_simulation;

 %Inizializzazione matlabFunction per assegnare valori alle variabili
 %simboliche
 ht = matlabFunction(F,'Optimize',false);
 ht1 = matlabFunction(D,'Optimize',false);
 ht2= matlabFunction(X_dot,'Optimize',false);
 ht3 = matlabFunction(H,'Optimize',false);
 ht4 = matlabFunction(h,'Optimize',false);
 log_EKF = [];

 %Dopo aver creato la struttura proseguo col costrire il ciclo for in cui
 %inserirò sia la parte di predizione che di correzione

 k = 0;
 for t = 0:dt:t_simulation
     k = k + 1;

     [x_hat,P] =
predict_EKF(x_hat,P,Q,dt,log_vars.tau_phi(k),log_vars.tau_psi(k),W,F,D,X_dot,ht,ht1,ht2
); %predizione
     [x_hat,P,e] = correct_EKF(x_hat, P,
R,log_vars.dx_sens(k),log_vars.phi_dot_sens(k),log_vars.psi_sens(k),log_vars.y_alpha(k)
,ht3,ht4,Mk,h,H); %correzione
  log_EKF.x_hat(k,1:6) = x_hat;
  log_EKF.P(k,:,:) = P;
  log_EKF.e(k,1:4) = e;

 end


 %predizione

 function  [x_hat,P] = predict_EKF(x_hat,P,Q,dt,tau_phi,tau_psi,W,F,D,X_dot,ht,ht1,ht2)
%Attraverso funzione feval e MatlabFunction assegno valori alle variabili
%simboliche
%Calcolo F, D e X_dot in x predetto al passo k/k e rumore nullo
 F = feval(ht,x_hat(3),x_hat(4),x_hat(5),x_hat(6),tau_phi,tau_psi,0,0);
 D = feval(ht1,x_hat(5));
 X_dot = feval(ht2,x_hat(3),x_hat(4),x_hat(5),x_hat(6),tau_phi,tau_psi,0,0);
  x_hat = x_hat + X_dot * dt ; %discretizzazione
    P = F*P*F' + D*Q*D'; %Calolo matrice P al passo k+1/k
 end

%Correzione

 function [x_hat,P,e] = correct_EKF(x_hat, P,
R,dx_sens,phi_dot_sens,psi_sens,y_alpha,ht3,ht4,Mk,h,H)
 %Calcolo H e h in x predetto di k / k-1 e rumore nullo
 H = feval(ht3,x_hat(1),x_hat(2));
 h = feval(ht4,x_hat(1),x_hat(2),x_hat(4),x_hat(5),0,0,0,0);
 e = [psi_sens;phi_dot_sens;dx_sens;y_alpha] - h';
```

```
 e(1) = atan2(sin(e(1)), cos(e(1)));
 e(2) = atan2(sin(e(2)), cos(e(2)));
 e(4) = atan2(sin(e(4)),cos(e(4))); %la differenza di angoli deve essere compresa tra -
pi e pi quindi sfrutto atan2
 %Applico stima BLUE
 S = vpa(H*P*H' + Mk*R*Mk',3); %comando vpa per effettuare rapporti poichè variabili
simboliche in forma razionale
 G = vpa(P*H'*inv(S),3); %G = guadagno di correzione
 x_hat = x_hat + G*e
 P = (eye(6) - G*H)*P*(eye(6) - G*H)' + G*Mk*R*Mk'*G'
  end
```

EKFWITHSMOOTHER.MAT

```
%L'unica differenza con EKF.mat è che ci serve la matrice F per lo smoother

t_simulation=10;
out=sim('isiprojectsimulink',t_simulation);
log_vars = [];
log_vars.X = (out.X)';
log_vars.Y = (out.Y)';
log_vars.dx_sens = (out.dx_sens)';
log_vars.phi_dot = (out.phi_dot)';
log_vars.phi_dot_sens = (out.phi_dot_sens)';
log_vars.psi = (out.psi)';
log_vars.psi_dot= (out.psi_dot)';
log_vars.psi_sens = (out.psi_sens)';
log_vars.tau_phi = (out.tau_phi)';
log_vars.tau_psi = (out.tau_psi)';
log_vars.theta = (out.theta)';
log_vars.y_alpha = (out.y_alpha)';
log_vars.x_hat = x_hat;
log_vars.P = P;


k=0;
 for p=0:dt:t_simulation
     k=k+1;
 log_vars.t(k)=p;
end

 log_vars.dt = t_sample;
 log_vars.t_simulation = t_simulation;

 ht = matlabFunction(F);
 ht1 = matlabFunction(D);
 ht2= matlabFunction(X_dot);
 ht3 = matlabFunction(H);
 ht4 = matlabFunction(h);
 log_EKF = [];

 %Dopo aver creato la struttura proseguo col costrire il ciclo for in cui
 %inserirò sia la parte di predizione che di correzione
```

```
 k = 0;
 for t = 0:dt:t_simulation
     k = k + 1;

     [x_hat,P,F] =
predict_EKF(x_hat,P,Q,dt,log_vars.tau_phi(k),log_vars.tau_psi(k),W,F,D,X_dot,ht,ht1,ht2
); %predizione
     log_filter.x_hat_prediction(k,1:6) = x_hat;
        log_filter.P_prediction(k,:,:) = P;
        log_filter.F_matrix(k,:,:) = F; %Estraggo matrice F che ci servirà per lo
smoother


     [x_hat,P,e] = correct_EKF(x_hat, P,
R,log_vars.dx_sens(k),log_vars.phi_dot_sens(k),log_vars.psi_sens(k),log_vars.y_alpha(k)
,ht3,ht4,Mk,h,H); %correzione
        log_filter.x_hat_correction(k,1:6) = x_hat;
        log_filter.P_correction(k,:,:) = P;

 end

 log_EKF = log_filter;
 save('EKF_smoother', 'log_EKF');

 %predizione

 function  [x_hat,P,F] =
predict_EKF(x_hat,P,Q,dt,tau_phi,tau_psi,W,F,D,X_dot,ht,ht1,ht2)

 F = feval(ht,x_hat(3),x_hat(4),x_hat(5),x_hat(6),tau_phi,tau_psi,0,0);
 D = feval(ht1,x_hat(5));
 X_dot = feval(ht2,x_hat(3),x_hat(4),x_hat(5),x_hat(6),tau_phi,tau_psi,0,0);
  x_hat = x_hat + X_dot * dt ;
    P = F*P*F' + D*Q*D';
 end

%Correzione

 function [x_hat,P,e] = correct_EKF(x_hat, P,
R,dx_sens,phi_dot_sens,psi_sens,y_alpha,ht3,ht4,Mk,h,H)
 H = feval(ht3,x_hat(1),x_hat(2));
 h = feval(ht4,x_hat(1),x_hat(2),x_hat(4),x_hat(5),0,0,0,0);
 e = [psi_sens;phi_dot_sens;dx_sens;y_alpha] - h';
 e(1) = atan2(sin(e(1)), cos(e(2)));
 e(4) = atan2(sin(e(4)),cos(e(4))); %la differenza di angoli deve essere compresa tra -
pi e pi
 S = vpa(H*P*H' + Mk*R*Mk',3);
 G = vpa(P*H'*inv(S),3);
 x_hat = x_hat + G*e
 P = (eye(6) - G*H)*P*(eye(6) - G*H)' + G*Mk*R*Mk'*G'
  end
```

RTSSMOOTHER.MAT

```
%La fase in avanti è stata fatta con l'EKF, in questo script ci occuperemo
%della fase all'indietro sfruttando le formule viste a teoria
%Dopo aver ottenuto tutte le x_hat e P dal filtro nella fase in avanti,
% esse serviranno per la fase a ritroso dove si sfrutteranno le formule conosciute a
teorie.
% A questo punto si calcolano le nuove x_hat e P a ritroso.

log_EKF.x_hat_correction = double(log_EKF.x_hat_correction);
%Inizializzazione ultime colonne uguali alle ultime x_hat e P in uscita dal filtro
esteso.

log_EKF.x_hat_smoothed(size(log_EKF.x_hat_prediction,1),:) =
log_EKF.x_hat_correction(size(log_EKF.x_hat_prediction,1),:);
log_EKF.P_smoothed(size(log_EKF.x_hat_prediction,1),:,:) =
[log_EKF.P_correction(size(log_EKF.x_hat_prediction,1),:,1);log_EKF.P_correction(size(l
og_EKF.x_hat_prediction,1),:,2);log_EKF.P_correction(size(log_EKF.x_hat_prediction,1),:
,3);log_EKF.P_correction(size(log_EKF.x_hat_prediction,1),:,4);log_EKF.P_correction(siz
e(log_EKF.x_hat_prediction,1),:,5);log_EKF.P_correction(size(log_EKF.x_hat_prediction,1
),:,6)];


   for k = size(log_EKF.x_hat_prediction,1)-1:-1:1 %k va da n-1 fino a 1 con step di 1

    % Ck = P_k|k * F_k+1' * P_k+1|k^(-1)
    Ck =
[log_EKF.P_correction(k,:,1);log_EKF.P_correction(k,:,2);log_EKF.P_correction(k,:,3);lo
g_EKF.P_correction(k,:,4);log_EKF.P_correction(k,:,5); ...
        log_EKF.P_correction(k,:,6)] ...
        * [log_EKF.F_matrix(k+1,:,1);
log_EKF.F_matrix(k+1,:,2);log_EKF.F_matrix(k+1,:,3);log_EKF.F_matrix(k+1,:,4);log_EKF.F
_matrix(k+1,:,5); ...
        log_EKF.F_matrix(k+1,:,6);]' / ...

[log_EKF.P_prediction(k+1,:,1);log_EKF.P_prediction(k+1,:,2);log_EKF.P_prediction(k+1,:
,3);log_EKF.P_prediction(k+1,:,4); ...
        log_EKF.P_prediction(k+1,:,5);log_EKF.P_prediction(k+1,:,6)];

    % x_k|n = x_k|k + Ck*(x_k+1|n - x_k+1|k)
      log_EKF.x_hat_smoothed(k,:) = log_EKF.x_hat_correction(k,:)' +
Ck*(log_EKF.x_hat_smoothed(k+1,:)' - log_EKF.x_hat_prediction(k+1,:)');


     % P_k|n = P_k|k + Ck*(P_k+1|n - P_k+1|k)*Ck'
    log_EKF.P_smoothed(k,:,:) =
[log_EKF.P_correction(k,:,1);log_EKF.P_correction(k,:,2);log_EKF.P_correction(k,:,3);
...

log_EKF.P_correction(k,:,4);log_EKF.P_correction(k,:,5);log_EKF.P_correction(k,:,6)]
...
         +
Ck*([log_EKF.P_smoothed(k+1,:,1);log_EKF.P_smoothed(k+1,:,2);log_EKF.P_smoothed(k+1,:,3
); ...

log_EKF.P_smoothed(k+1,:,4);log_EKF.P_smoothed(k+1,:,5);log_EKF.P_smoothed(k+1,:,6)] -
[log_EKF.P_prediction(k+1,:,1); ...
```

```
log_EKF.P_prediction(k+1,:,2);log_EKF.P_prediction(k+1,:,3);log_EKF.P_prediction(k+1,:,
4);log_EKF.P_prediction(k+1,:,5); ...
         log_EKF.P_prediction(k+1,:,6)])*Ck';
  end
```

EKFDIFFPERIODI.MAT

```
%In questo script verrà implementato il filtro di Kalman però cosiderando i
%periodi di campionamento dei sensori diversi
%I dati in uscita da simulink sono di tipo Timeseries

t_simulation=10;
out=sim('isiprojectsimulink2',t_simulation);
log_vars = [];
log_vars.X = (out.X)';
log_vars.Y = (out.Y)';
log_vars.dx_sens = (out.dx_sens)';
log_vars.phi_dot = (out.phi_dot)';
log_vars.phi_dot_sens = (out.phi_dot_sens)';
log_vars.psi = (out.psi)';
log_vars.psi_dot= (out.psi_dot)';
log_vars.psi_sens = (out.psi_sens)';
log_vars.tau_phi = (out.tau_phi)';
log_vars.tau_psi = (out.tau_psi)';
log_vars.theta = (out.theta)';
log_vars.y_alpha = (out.y_alpha)';
log_vars.x_hat = x_hat;
log_vars.P = P;

k=0;
 for p=0:dt:t_simulation
     k=k+1;
 log_vars.t(k)=p;
end

 log_vars.dt = t_sample;
 log_vars.t_simulation = t_simulation;

 ht = matlabFunction(F);
 ht1 = matlabFunction(D);
 ht2= matlabFunction(X_dot);
 ht3 = matlabFunction(H);
 ht4 = matlabFunction(h);
 log_EKF = [];

 %Inizializzo vettore delle misure utilizzate che conterrà tutti i tempi
 %delle misure utilizzate essendo univoci

 misure_utilizzate = [];
 misure_psi = 0;
 misure_phi_dot = 0;
 misure_dx = 0;

 %Sto aggiungendo delle misure poichè ci saranno alcuni sensori che avranno
 %più misure e altri sensori che avranno meno misure, queste misure aggiunte
 % non sono veritiere perciò l'errore alla fine aumenterà.
prova1 = size(log_vars.y_alpha.Time,1) - size(log_vars.phi_dot_sens.Time,1);
```

```
prova2 = size(log_vars.y_alpha.Time,1) - size(log_vars.dx_sens.Time,1);
prova3 = size(log_vars.y_alpha.Time,1) - size(log_vars.psi_sens.Time,1);

for t = 1 : 1 : prova1
    log_vars.phi_dot_sens = addsample(log_vars.phi_dot_sens,'Data',0,'Time',10);
end

for t = 1 : 1 : prova2
    log_vars.dx_sens = addsample(log_vars.dx_sens,'Data',0,'Time',10);
end

for t = 1 : 1 : prova3
    log_vars.psi_sens = addsample(log_vars.psi_sens,'Data',0,'Time',10);
end

%Inizializzo matrice dei flag che mi servirà per andare a togliere eventuali
%colonne o righe alle matrici

flag = zeros(size(log_vars.t,2),4);
  k = 0;
 for t = 0:dt:t_simulation
     k = k + 1;
     T(k) = t;
     [x_hat,P] =
predict_EKF(x_hat,P,Q,dt,log_vars.tau_phi(k),log_vars.tau_psi(k),W,F,D,X_dot,ht,ht1,ht2
); %predizione

      if T(k) < log_vars.psi_sens.Time(k)
          if ismember(misure_utilizzate.psi,log_vars.psi_sens.time(k-1)) ~= 0 %mi dice
se la misura precedente è stata utilizzata poichè va a controllare se il tempo della
misura precedente appartiene al vettore delle misure utilizzate
           flag(k,1) = 1;
           misure_psi = log_vars.psi_sens.time(k-1);
      else
          misure_psi = log_vars.psi_sens.time(k-1);%se non è stata utilizzata allora la
utilizzo
      %   log_vars.psi_sens.Data(k) = log_vars.psi_sens.Data(k-1);
          flag(k,1) = 0;
          end
      end

     if t < log_vars.phi_dot_sens.Time(k)
         if ismember(misure_utilizzate.phi_dot,log_vars.phi_dot_sens.time(k-1)) ~= 0
           flag(k,2) = 1;
           misure_phi_dot = log_vars.phi_dot_sens.time(k-1);
      else
          misure_phi_dot = log_vars.phi_dot_sens.time(k-1);
          flag(k,2) = 0;
       %   log_vars.phi_dot_sens.Data(k) = log_vars.phi_dot_sens.Data(k-1);
         end
     end

     if t < log_vars.dx_sens.Time(k)
         if ismember(misure_utilizzate.dx,log_vars.dx_sens.Time(k-1)) ~= 0
           flag(k,4) = 1;
           misure_dx= log_vars.dx_sens.Time(k-1);
      else
         misure_dx = log_vars.dx_sens.Time(k-1);
         flag(k,4) = 0;
     %    log_vars.dx_sens.Data(k) = log_vars.dx_sens.Data(k-1);
      end
```

```
    end

   [x_hat,P,e] = correct_EKF(x_hat, P,
R,log_vars.dx_sens.Data(k),log_vars.phi_dot_sens.Data(k),log_vars.psi_sens.Data(k),log_
vars.y_alpha.Data(k),ht3,ht4,Mk,h,H,flag,k); %correzione
  log_EKF.x_hat(k,1:6) = x_hat;
  log_EKF.P(k,:,:) = P;
  misure_utilizzate.psi(k) = misure_psi;
  misure_utilizzate.phi_dot(k) = misure_phi_dot;
  misure_utilizzate.dx(k) = misure_dx;

 end


 %predizione

 function  [x_hat,P] = predict_EKF(x_hat,P,Q,dt,tau_phi,tau_psi,W,F,D,X_dot,ht,ht1,ht2)

 F = feval(ht,x_hat(3),x_hat(4),x_hat(5),x_hat(6),tau_phi,tau_psi,0,0);
 D = feval(ht1,x_hat(5));
 X_dot = feval(ht2,x_hat(3),x_hat(4),x_hat(5),x_hat(6),tau_phi,tau_psi,0,0);
  x_hat = x_hat + X_dot * dt ;
    P = F*P*F' + D*Q*D';
 end

%Correzione
%Quando flag = 0 le misure le prende invece quando flag = 1 le misure non
%le prende, se la misura è stata utilizzata oppure non è arrivata allora le
%matrici perderanno righe o/e colonne come posssiamo notare dagli if
 function [x_hat,P,e] = correct_EKF(x_hat, P,
R,dx_sens,phi_dot_sens,psi_sens,y_alpha,ht3,ht4,Mk,h,H,flag,k)
 H = feval(ht3,x_hat(1),x_hat(2));
 h = feval(ht4,x_hat(1),x_hat(2),x_hat(4),x_hat(5),0,0,0,0);
 misure = [psi_sens ; phi_dot_sens ; dx_sens ; y_alpha];
 if flag(k,1) == 1
     H(1,:) = [];
     h(1) = [];
     R(1,:) = [];
     R(:,1) = [];
     Mk(1,:) = [];
     Mk(:,1) = [];
     misure(1) = [];
 end
  if flag(k,2) == 1
     H(2,:) = [];
     h(2) = [];
     R(2,:) = [];
     R(:,2) = [];
     Mk(2,:) = [];
     Mk(:,2) = [];
     misure(2) = [];
  end
  if flag(k,3) == 1
     H(3,:) = [];
     h(3) = [];
     R(3,:) = [];
     R(:,3) = [];
     Mk(3,:) = [];
     Mk(:,3) = [];
     misure(3) = [];
 end
```

```
 e = misure - h';
 S = vpa(H*P*H' + Mk*R*Mk',3);
 G = vpa(P*H'*inv(S),3);
 x_hat = x_hat + G*e
 P = (eye(6) - G*H)*P*(eye(6) - G*H)' + G*Mk*R*Mk'*G'
 end
```

PLOTMAT.MAT

```
t_simulation=10;
out=sim('isiprojectsimulink',t_simulation);
log_vars = [];
log_vars.X = (out.X)';
log_vars.Y = (out.Y)';
log_vars.dx_sens = (out.dx_sens)';
log_vars.phi_dot = (out.phi_dot)';
log_vars.phi_dot_sens = (out.phi_dot_sens)';
log_vars.psi = (out.psi)';
log_vars.psi_dot= (out.psi_dot)';
log_vars.psi_sens = (out.psi_sens)';
log_vars.tau_phi = (out.tau_phi)';
log_vars.tau_psi = (out.tau_psi)';
log_vars.theta = (out.theta)';
log_vars.y_alpha = (out.y_alpha)';
log_vars.x_hat = x_hat;
log_vars.P = P;


k=0;
 for p=0:dt:t_simulation
     k=k+1;
 log_vars.t(k)=p;
end

 log_vars.dt = t_sample;
 log_vars.t_simulation = t_simulation;


figure
plot(out.X,out.Y)
```

MODELO_ROBOT_DIFERENCIAL3.MAT

```
% CURSOS ONLINE (MI PLATAFORMA RECOMENDADO)
%
% Robótica con Matlab y Arduino: Modelo y Simulación: http://bit.ly/2MHWYPb
% Robótica con Matlab y Arduino: Diseño de controladores: http://bit.ly/2SXaDW3
%
% clc
% clear
% close all
```

```
t=0:t_simulation/dt;

u=0.1*ones(1,length(t));
w=0.08*ones(1,length(t));


xr(1)=log_vars.X(1);
yr(1)=log_vars.Y(1);
phi(1)=log_vars.theta(1);



pasos=1;  fig=figure;
set(fig,'position',[10 60 980 600]);
axis square; cameratoolbar
axis([-20 20 -20 20 -0.5 1]); grid on
MobileRobot;
M1=MobilePlot(xr(1),yr(1),phi(1));hold on
M2=plot(xr(1),yr(1),'b','LineWidth',2);


for i=1:pasos:length(t)


    delete (M1)
    delete (M2)
    M1=MobilePlot(log_vars.X(1,i),log_vars.Y(1,i),log_vars.theta(1,i)); hold on
    M2=plot(log_vars.X(1:i),log_vars.Y(1:i),'blue','LineWidth',2);

    pause(dt)

end
```

MOBILEROBOT.MAT

```
function MobileRobot
width=1.2;
lth=1;
high=0.3;
radio=0.25;

global Mobile;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% Cuerpo %%%%%%%%%%%%%%%%%%%%%%%%

base=[19.35 -12.39 0,
18.366 -14.764 0,
16.802 -16.802 0,
14.764 -18.366 0,
12.39 -19.35 0,
9.843 -19.685 0,
9.843 -14.764 0,
-9.843 -14.764 0,
-9.843 -19.685 0,
```

```
-12.39 -19.35 0,
-14.764 -18.366 0,
-16.802 -16.802 0,
-18.366 -14.764 0,
-19.35 -12.39 0,
-19.685 -9.843 0,
-19.685 9.843 0,
-19.35 12.39 0,
-18.366 14.764 0,
-16.802 16.802 0,
-14.764 18.366 0,
-12.39 19.35 0,
-9.843 19.685 0,
-9.843 14.764 0,
9.843 14.764 0,
9.843 19.685 0,
12.39 19.35 0,
14.764 18.366 0,
16.802 16.802 0,
18.366 14.764 0,
19.35 12.39 0,
19.685 9.843 0,
18.973 8.297 0,
18.384 6.699 0,
17.922 5.061 0,
17.59 3.391 0,
17.39 1.701 0,
17.323 -0 0,
17.39 -1.701 0,
17.59 -3.391 0,
17.922 -5.061 0,
18.384 -6.699 0,
18.973 -8.297 0,
19.685 -9.843 0]/39.37;

base(:,1)=width*base(:,1);
base(:,2)=lth*base(:,2);

zs=high;

for k=1:length(base)-1
    Mobile.Base{k}=[base(k,1) base(k,2) base(k,3) 1;base(k+1,1) base(k+1,2) base(k,3)
1;base(k+1,1)  base(k+1,2) zs 1 ;base(k,1) base(k,2) zs  1]';
if k==length(base)-1
    Mobile.Base{k+1}=[base(length(base),1) base(length(base),2) base(1,3) 1; base(1,1)
base(1,2) base(1,3) 1;base(1,1)  base(1,2) zs 1;base(length(base),1)
base(length(base),2) zs 1]';
end
end

Mobile.Base{k+2}=[base'; ones(1,length(base))];
Mobile.Base{k+3}=[[base(:,1) base(:,2) base(:,3)+zs]'; ones(1,length(base))];

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% LLANTAS %%%%%%%%%%%%%%%%%%%%%%%%%
th=0:0.3:2*pi;
x=radio*cos(th);
z=radio*sin(th);
y=zeros(1,length(th));

wheel=[x' y' z'+high/2];
```

```
zs=0.1;

for k=1:length(wheel)-1
    Mobile.Wheel{k}=[wheel(k,1) wheel(k,2) wheel(k,3) 1;wheel(k+1,1) wheel(k,2)
wheel(k+1,3) 1;wheel(k+1,1) zs wheel(k+1,3) 1 ;wheel(k,1)  zs wheel(k,3)  1]';
if k==length(wheel)-1
    Mobile.Wheel{k+1}=[wheel(length(wheel),1) wheel(length(wheel),2)
wheel(length(wheel),3) 1; wheel(1,1) wheel(length(wheel),2) wheel(1,3) 1;wheel(1,1)  zs
wheel(1,3)  1;wheel(length(wheel),1) zs wheel(length(wheel),3)  1]';
end
end

Mobile.Wheel{k+2}=[wheel'; ones(1,length(wheel))];
Mobile.Wheel{k+3}=[[wheel(:,1) wheel(:,2)+zs wheel(:,3)]'; ones(1,length(wheel))];
```

MOBILEPLOT.MAT

```
function  Mobile_Graph=MobilePlot(dx,dy,angz)
global Mobile;
dz=0;

% Matriz de rotación z
Rz=[ cos(angz), -sin(angz) 0 0; sin(angz) cos(angz) 0 0; 0 0 1 0;0 0 0 1];
Rot_TheWholeArm=Rz;
Rot_TheWholeArm(1:3,4) = [dx dy dz]';

WheelColor='g';
BodyColor=[0 0 0.5];
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% CUERPO %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
tam=0;
for ii = 1:(length(Mobile.Base))
    robotPatch = Rot_TheWholeArm*Mobile.Base{ii};
    Mobile_Graph(tam+ii) =
patch(robotPatch(1,:),robotPatch(2,:),robotPatch(3,:),BodyColor,'LineWidth',1.8);
end

Rot_TheWholeArm(1:3,4) = [dx-0.38*sin(angz) dy+0.38*cos(angz) dz]';

tam=tam+ii;

for ii =1:(length(Mobile.Wheel))
    robotPatch = Rot_TheWholeArm*Mobile.Wheel{ii};
    Mobile_Graph(tam+ii) =
patch(robotPatch(1,:),robotPatch(2,:),robotPatch(3,:),WheelColor,'LineWidth',1.8);
end

Rot_TheWholeArm(1:3,4) = [dx+((0.38+0.1)*sin(angz)) dy-((0.38+0.1)*cos(angz)) dz]';

tam=tam+ii;
for ii =1:(length(Mobile.Wheel))
    robotPatch = Rot_TheWholeArm*Mobile.Wheel{ii};
    Mobile_Graph(tam+ii) =
patch(robotPatch(1,:),robotPatch(2,:),robotPatch(3,:),WheelColor,'LineWidth',1.8);
end
```

PLOTTINOEKF.MAT

```
figure(1)
clf
grid on
box on
title('position estimation error [m]')


log_vars.x = double(log_EKF.x_hat(:,1)');
log_vars.y = double(log_EKF.x_hat(:,2)');
log_vars.thetas = double(log_EKF.x_hat(:,3)');
log_vars.phi_dots=double(log_EKF.x_hat(:,4)');
log_vars.psis=double(log_EKF.x_hat(:,5)');
log_vars.psi_dots=double(log_EKF.x_hat(:,6)');

figure(1)
plot(log_vars.t, vecnorm([log_vars.X ; log_vars.Y] - [log_vars.x ; log_vars.y]));
hold on

figure(2)
x = log_vars.t;


tiledlayout(3,2)
nexttile
y1= log_vars.X;
y2 = log_vars.x;
hold on
plot(x,y1)
plot(x,y2)
xlabel('tempo')
ylabel('x')
legend('posizione x vera','posizione x predetta')
hold off

nexttile
y7=log_vars.phi_dot;
y8=log_vars.phi_dots;
hold on
plot(x,y7)
plot(x,y8)
xlabel('tempo')
ylabel('phi_dot')
legend('posizione phi_dot vera','posizione phi_dot predetta')
hold off

nexttile
y3=log_vars.Y;
y4=log_vars.y;
hold on
plot(x,y3)
plot(x,y4)
xlabel('tempo')
ylabel('y')
legend('posizione y vera','posizione y predetta')
hold off
```

```
nexttile
y9=log_vars.psi;
y10=log_vars.psis;
hold on
plot(x,y9)
plot(x,y10)
xlabel('tempo')
ylabel('psi')
legend('posizione psi vera','posizione psi predetta')
hold off

nexttile
y5=log_vars.theta;
y6=log_vars.thetas;
hold on
plot(x,y5)
plot(x,y6)
xlabel('tempo')
ylabel('theta')
legend('posizione theta vera','posizione theta predetta')
hold off


nexttile
y11=log_vars.psi_dot;
y12=log_vars.psi_dots;
hold on
plot(x,y11)
plot(x,y12)
xlabel('tempo')
ylabel('psi_dot')
legend('posizione psi_dot vera','posizione psi_dot predetta')
hold off

figure(3)


tiledlayout(3,2)
nexttile
y1= log_vars.X;
y2 = log_vars.x;
hold on
plot(x,y1-y2)
title('X vera - X predetta')
hold off

nexttile
y7=log_vars.phi_dot;
y8=log_vars.phi_dots;
hold on
plot(x,y7-y8)

title('phi vero - phi predetto')
hold off

nexttile
y3=log_vars.Y;
y4=log_vars.y;
hold on
plot(x,y3-y4)
title('Y vera - Y predetta')
```

```
hold off

nexttile
y9=log_vars.psi;
y10=log_vars.psis;
hold on
plot(x,y9-y10)
title('psi vero - psi predetto')
hold off

nexttile
y5=log_vars.theta;
y6=log_vars.thetas;
hold on
plot(x,y5-y6)
title('theta vero - theta predetto')
hold off


nexttile
y11=log_vars.psi_dot;
y12=log_vars.psi_dots;
hold on
plot(x,y11-y12)
title('psi_dot vero - psi_dot predetto')
hold off
```

UKF_FILTER.MAT

```
%Filtro UKF

t_simulation=10;
out=sim('isiprojectsimulink',t_simulation);
log_vars = [];
log_vars.X = (out.X)';
log_vars.Y = (out.Y)';
log_vars.dx_sens = (out.dx_sens)';
log_vars.phi_dot = (out.phi_dot)';
log_vars.phi_dot_sens = (out.phi_dot_sens)';
log_vars.psi = (out.psi)';
log_vars.psi_dot= (out.psi_dot)';
log_vars.psi_sens = (out.psi_sens)';
log_vars.tau_phi = (out.tau_phi)';
log_vars.tau_psi = (out.tau_psi)';
log_vars.theta = (out.theta)';
log_vars.y_alpha = (out.y_alpha)';
log_vars.x_hat = x_hat;
log_vars.P = P;


k=0;
 for p=0:dt:t_simulation
     k=k+1;
 log_vars.t(k)=p;
end
```

```
 log_vars.dt = t_sample;
 log_vars.t_simulation = t_simulation;

 ht= matlabFunction(X_dot);
 ht1 = matlabFunction(h);

 k = 0;
 for t = 0:dt:t_simulation
     k = k + 1;

      [x_hat,P] = predict_UKF(x_hat,P,Q,dt,log_vars.tau_phi(k),log_vars.tau_psi(k),ht);
%predizione
      [x_hat,P,e] = correct_UKF(x_hat, P,
R,log_vars.dx_sens(k),log_vars.phi_dot_sens(k),log_vars.psi_sens(k),log_vars.y_alpha(k)
,ht1); %correzione
   log_UKF.x_hat(k,1:6) = x_hat;
   log_UKF.P(k,1:6,1:6) = P;
   log_UKF.e(k,1:4) = e;

  end


 function [x_hat,P] = predict_UKF(x_hat,P,Q,dt,tau_phi,tau_psi,ht)
 %In input alla trasformata Unscented devono andare le medie, le matrici di
 %covarianze e il modello non lineare. Le medie dei rumori sono nulle
    [x_hat,P] = UnscentedTransform_F_Function([x_hat; 0; 0], blkdiag(P, Q), dt,
tau_phi, tau_psi,ht);
 end

function [x_hat,P,e] = correct_UKF(x_hat, P,
R,dx_sens,phi_dot_sens,psi_sens,y_alpha,ht1)
 [hat_y, S, Pxy] = UnscentedTransform_H_Function(x_hat, P,ht1);
 %essendo il rumore additivo non va nelle medie in input alla trasformata
 %Unscented poichè verrrà sommato dopo
    y = [psi_sens; phi_dot_sens; dx_sens ;y_alpha];
    S = S + R; %sommiamo la matrice delle varianze dei rumori di misura
    e = y - hat_y;
   e(1) = atan2(sin(e(1)),cos(e(2)));
   e(4) = atan2(sin(e(4)),cos(e(4))); %la differenza di angoli deve essere compresa tra
-pi e pi
    L = Pxy*S^(-1);

    x_hat = x_hat + L*e;
    P = P - L*S*L';
end

 function [propagated_mean, propagated_cov] =
UnscentedTransform_F_Function(misura_priori, cov_priori, dt, tau_phi, tau_psi,ht)
    % definizione parametri
    alpha = 1;
    beta = 2;             % ottimo nel caso gaussiano
    kappa = 0;            % lambda = 0 (peso centrale dei sigma point)

    enne = size(misura_priori,1); %n è il vettore iniziale composto dalle variabili di
stato e la media dei rumori che è nulla(nel nostro caso = 8)

    lambda = alpha^2*(enne + kappa) - enne;

    % Calcolo dei pesi
    w0 = lambda/(lambda + enne);
    wc(1) = w0 + 1 - alpha^2 + beta;
```

```
    wc(2:2*enne+1) = 1/2/(enne+lambda);

    wm = wc;
    wm(1) = w0;

    % Fattorizzazione matrice delle covarianze
    % SVD
    [U,S,~] = svd(cov_priori);
    GAMMA = U*S^(1/2); %le colonne ci indicano le direzioni per andare a generare i
sigma point

    % generazione degli 2n+1 sigma point (saranno 17 sigma point)
    sigma_points = misura_priori; % il primo sigma point centrale coicide con la media
    for i = 1:size(GAMMA,2) %definisco gli altri 2n sigma point simmetrici centrati
sulla media
        sigma_points(:,i+1)      = misura_priori + sqrt(enne+lambda)*GAMMA(:,i);
        sigma_points(:,i+1+enne) = misura_priori - sqrt(enne+lambda)*GAMMA(:,i);
    end

    %Per ognuno di questi sigma-point applichiamo la funzione f in modo
    %tale da ottenere i sigma-point propagati con il modello del sistema
    for i = 1:size(sigma_points,2)
        X_dot =
feval(ht,sigma_points(3,i),sigma_points(4,i),sigma_points(5,i),sigma_points(6,i),tau_ph
i,tau_psi,sigma_points(7,i),sigma_points(8,i));
        propagated_sigma_points(1:6,i) = sigma_points(1:6,i) + dt*X_dot;

    end

    %Media pesata dai wn (fare attenzione alla media di angoli: si sfrutta
    %l'atan2
   propagated_mean(1,:) = propagated_sigma_points(1,:)*wm';
   propagated_mean(2,:) = propagated_sigma_points(2,:)*wm';
   propagated_mean(3,:) =
atan2(sin(propagated_sigma_points(3,:)*wm'),cos(propagated_sigma_points(3,:)*wm'));
   propagated_mean(4,:) =
atan2(sin(propagated_sigma_points(4,:)*wm'),cos(propagated_sigma_points(4,:)*wm'));
   propagated_mean(6,:) =
atan2(sin(propagated_sigma_points(6,:)*wm'),cos(propagated_sigma_points(6,:)*wm'));
   propagated_mean(5,:) =
atan2(sin(propagated_sigma_points(5,:)*wm'),cos(propagated_sigma_points(5,:)*wm'));

     % Tilde propagato (differenza rispetto alla media che ci servirà per
     % la matrice di covarianza campionaria)
     propagated_tilde(1,:) = propagated_sigma_points(1,:) - propagated_mean(1,:);
     propagated_tilde(2,:) = propagated_sigma_points(2,:) - propagated_mean(2,:);
     propagated_tilde(3,:) = atan2(sin(propagated_sigma_points(3,:) -
propagated_mean(3,:)),cos(propagated_sigma_points(3,:) - propagated_mean(3,:)));
     propagated_tilde(4,:) = atan2(sin(propagated_sigma_points(4,:) -
propagated_mean(4,:)),cos(propagated_sigma_points(4,:) - propagated_mean(4,:)));
     propagated_tilde(5,:) = atan2(sin(propagated_sigma_points(5,:) -
propagated_mean(5,:)),cos(propagated_sigma_points(5,:) - propagated_mean(5,:)));
     propagated_tilde(6,:) = atan2(sin(propagated_sigma_points(6,:) -
propagated_mean(6,:)),cos(propagated_sigma_points(6,:) - propagated_mean(6,:)));


    % Momento incrociato non ci serve
    propagated_cov = zeros(6,6);
    for i = 1:size(sigma_points,2)
        propagated_cov = propagated_cov +
wc(i)*propagated_tilde(:,i)*propagated_tilde(:,i)';
```

```
    end
 end


function [virtual_measurements_mean, virtual_measurements_cov, cross_cov] =
UnscentedTransform_H_Function(prior_mean, prior_cov,ht1)

    alpha = 1;
    beta = 2;
    kappa = 0;

    enne = size(prior_mean,1); % n = 6

    lambda = alpha^2*(enne + kappa) - enne;

    w0 = lambda/(lambda + enne);
    wc(1) = w0 + 1 - alpha^2 + beta;
    wc(2:2*enne+1) = 1/2/(enne+lambda);

    wm = wc;
    wm(1) = w0;

    [U,S,~] = svd(prior_cov);
    GAMMA = U*S^(1/2);

    sigma_points = prior_mean;
    for i = 1:size(GAMMA,2)
        sigma_points(:,i+1)      = prior_mean + sqrt(enne+lambda)*GAMMA(:,i);
        sigma_points(:,i+1+enne) = prior_mean - sqrt(enne+lambda)*GAMMA(:,i);
    end

    % In questo caso bisogna propagare i sigma points con il modello di
    % osservazione
    for i = 1:size(sigma_points,2)
        propagated_sigma_points(1:4,i) =
feval(ht1,sigma_points(1,i),sigma_points(2,i),sigma_points(4,i),sigma_points(5,i),0,0,0
,0);
    end


    virtual_measurements_mean(1,:) =
atan2(sin(propagated_sigma_points(1,:)*wm'),cos(propagated_sigma_points(1,:)*wm'));
    virtual_measurements_mean(2,:) =
atan2(sin(propagated_sigma_points(2,:)*wm'),cos(propagated_sigma_points(2,:)*wm'));
    virtual_measurements_mean(3,:) = propagated_sigma_points(3,:)*wm';
    virtual_measurements_mean(4,:) =
atan2(sin(propagated_sigma_points(4,:)*wm'),cos(propagated_sigma_points(4,:)*wm'));

    virtual_measurements_tilde(1,:) = atan2(sin(propagated_sigma_points(1,:) -
virtual_measurements_mean(1,:)),cos(propagated_sigma_points(1,:) -
virtual_measurements_mean(1,:)));
    virtual_measurements_tilde(2,:) = atan2(sin(propagated_sigma_points(2,:) -
virtual_measurements_mean(2,:)),cos(propagated_sigma_points(2,:) -
virtual_measurements_mean(2,:)));
    virtual_measurements_tilde(3,:) = propagated_sigma_points(3,:) -
virtual_measurements_mean(3,:);
    virtual_measurements_tilde(4,:) = atan2(sin(propagated_sigma_points(4,:) -
virtual_measurements_mean(4,:)),cos(propagated_sigma_points(4,:) -
virtual_measurements_mean(4,:)));
```

```
    %In questo caso il momento incrociato ci serve
    virtual_measurements_cov = zeros(4,4);
    cross_cov = zeros(6,4);
    for i = 1:size(sigma_points,2)
        virtual_measurements_cov = virtual_measurements_cov +
wc(i)*virtual_measurements_tilde(:,i)*virtual_measurements_tilde(:,i)';
        cross_cov = cross_cov + wc(i)*(sigma_points(:,i) -
prior_mean)*virtual_measurements_tilde(:,i)';
    end
end
```

UKF_FILTER_DIFF_PERIODI.MAT

```
%Filtro UKF con periodi di campionamento dei filtri diversi tra loro e diverso col
periodo di campionamento
%del filtro, stessa logica dell'EKF

t_simulation=10;
out=sim('isiprojectsimulink2',t_simulation);
log_vars = [];
log_vars.X = (out.X)';
log_vars.Y = (out.Y)';
log_vars.dx_sens = (out.dx_sens)';
log_vars.phi_dot = (out.phi_dot)';
log_vars.phi_dot_sens = (out.phi_dot_sens)';
log_vars.psi = (out.psi)';
log_vars.psi_dot= (out.psi_dot)';
log_vars.psi_sens = (out.psi_sens)';
log_vars.tau_phi = (out.tau_phi)';
log_vars.tau_psi = (out.tau_psi)';
log_vars.theta = (out.theta)';
log_vars.y_alpha = (out.y_alpha)';
log_vars.x_hat = x_hat;
log_vars.P = P;


k=0;
 for p=0:dt:t_simulation
     k=k+1;
 log_vars.t(k)=p;
end

 log_vars.dt = t_sample;
 log_vars.t_simulation = t_simulation;

 ht= matlabFunction(X_dot);
 ht1 = matlabFunction(h);

 misure_utilizzate = [];
 misure_psi = 0;
 misure_phi_dot = 0;
 misure_dx = 0;

 for t = 1 : 1 : 1
    log_vars.phi_dot_sens = addsample(log_vars.phi_dot_sens,'Data',0,'Time',10);
end
```

```
for t = 1 : 1 : 1
    log_vars.dx_sens = addsample(log_vars.dx_sens,'Data',0,'Time',10);
end

for t = 1 : 1 : 1
    log_vars.psi_sens = addsample(log_vars.psi_sens,'Data',0,'Time',10);
end

 k = 0;
 flag = zeros(size(log_vars.t,2),4);

 for t = 0:dt:t_simulation
     k = k + 1;
      T(k) = t;
      [x_hat,P] = predict_UKF(x_hat,P,Q,dt,log_vars.tau_phi(k),log_vars.tau_psi(k),ht);
%predizione

      if T(k) < log_vars.psi_sens.Time(k)
          if ismember(misure_utilizzate.psi,log_vars.psi_sens.time(k-1)) ~= 0 %mi dice
se la misura precedente è stata utilizzata
           flag(k,1) = 1;
           misure_psi = log_vars.psi_sens.time(k-1);
      else
          misure_psi = log_vars.psi_sens.time(k-1);%se non è stata utilizzata allora la
utilizzo
      %   log_vars.psi_sens.Data(k) = log_vars.psi_sens.Data(k-1);
          flag(k,1) = 0;
          end
      end

     if t < log_vars.phi_dot_sens.Time(k)
         if ismember(misure_utilizzate.phi_dot,log_vars.phi_dot_sens.time(k-1)) ~= 0
           flag(k,2) = 1;
           misure_phi_dot = log_vars.phi_dot_sens.time(k-1);
      else
          misure_phi_dot = log_vars.phi_dot_sens.time(k-1);
          flag(k,2) = 0;
       %   log_vars.psi_sens.Data(k) = log_vars.psi_sens.Data(k-1);
         end
     end

     if t < log_vars.dx_sens.Time(k)
         if ismember(misure_utilizzate.dx,log_vars.dx_sens.Time(k-1)) ~= 0
           flag(k,4) = 1;
           misure_dx= log_vars.dx_sens.Time(k-1);
      else
         misure_dx = log_vars.dx_sens.Time(k-1);
         flag(k,4) = 0;
       %  log_vars.dx_sens.Data(k) = log_vars.dx_sens.Data(k-1);
      end
  end

      [x_hat,P,e] = correct_UKF(x_hat, P,
R,log_vars.dx_sens.Data(k),log_vars.phi_dot_sens.Data(k),log_vars.psi_sens.Data(k),log_
vars.y_alpha.Data(k),ht1,flag,k); %correzione
   log_UKF.x_hat(k,1:6) = x_hat;
   log_UKF.P(k,1:6,1:6) = P;
   %log_UKF.e(k,1:4) = e;
  misure_utilizzate.psi(k) = misure_psi;
  misure_utilizzate.phi_dot(k) = misure_phi_dot;
  misure_utilizzate.dx(k) = misure_dx;
```

```
  end


 function [x_hat,P] = predict_UKF(x_hat,P,Q,dt,tau_phi,tau_psi,ht)
    [x_hat,P] = UnscentedTransform_F_Function([x_hat; 0; 0], blkdiag(P, Q), dt,
tau_phi, tau_psi,ht);
 end

function [x_hat,P,e] = correct_UKF(x_hat, P,
R,dx_sens,phi_dot_sens,psi_sens,y_alpha,ht1,flag,k)
 [hat_y, S, Pxy] = UnscentedTransform_H_Function(x_hat, P,ht1);
    y = [psi_sens; phi_dot_sens; dx_sens ;y_alpha];

    if flag(k,1) == 1
     y(1) = [];
     hat_y(1) = [];
     R(1,:) = [];
     R(:,1) = [];
     S(1,:) = [];
     S(:,1) = [];
     Pxy(:,1) = [];
 end
  if flag(k,2) == 1
      y(2) = [];
      hat_y(2) = [];
       R(2,:) = [];
       R(:,2) = [];
       S(2,:) = [];
       S(:,2) = [];
       Pxy(:,2) = [];
  end
  if flag(k,3) == 1
     y(3) = [];
     hat_y(3) = [];
      R(3,:) = [];
      R(:,3) = [];
      S(3,:) = [];
      S(:,3) = [];
      Pxy(:,3) = [];
 end
    S = S + R;
    e = y - hat_y;
   %e(1) = atan2(sin(e(1)),cos(e(2)));
   %e(4) = atan2(sin(e(4)),cos(e(4))); %la differenza di angoli deve essere compresa
tra -pi e pi
    L = Pxy*S^(-1);

    x_hat = x_hat + L*e;
    P = P - L*S*L';
end

 function [propagated_mean, propagated_cov] =
UnscentedTransform_F_Function(misura_priori, cov_priori, dt, tau_phi, tau_psi,ht)
    % definizione parametri
    alpha = 1;            % no scalare
    beta = 2;             % ottimo nel caso gaussiano
    kappa = 0;            % lambda = 0 (peso centrale dei sigma point)

    enne = size(misura_priori,1); %n è il vettore iniziale composto dalle variabili di
stato e la media dei rumori che è nulla(nel nostro caso = 8)
```

```
    lambda = alpha^2*(enne + kappa) - enne;

    % Calcolo dei pesi
    w0 = lambda/(lambda + enne);
    wc(1) = w0 + 1 - alpha^2 + beta;
    wc(2:2*enne+1) = 1/2/(enne+lambda);

    wm = wc;
    wm(1) = w0;

    % Fattorizzazione matrice delle covarianze
    % SVD
    [U,S,~] = svd(cov_priori);
    GAMMA = U*S^(1/2); %le colonne ci indicano le direzioni per andare a generare i
sigma point

    % generazione degli 2n+1 sigma point (saranno 17 sigma point)
    sigma_points = misura_priori; % il primo sigma point centrale coicide con la media
    for i = 1:size(GAMMA,2) %definisco gli altri 2n sigma point simmetrici centrati
sulla media
        sigma_points(:,i+1)      = misura_priori + sqrt(enne+lambda)*GAMMA(:,i);
        sigma_points(:,i+1+enne) = misura_priori - sqrt(enne+lambda)*GAMMA(:,i);
    end

    %Per ognuno di questi sigma-point applichiamo la funzione f in modo
    %tale da ottenere i sigma-point propagati con il modello del sistema
    for i = 1:size(sigma_points,2)
        X_dot =
feval(ht,sigma_points(3,i),sigma_points(4,i),sigma_points(5,i),sigma_points(6,i),tau_ph
i,tau_psi,sigma_points(7,i),sigma_points(8,i));
        propagated_sigma_points(1:6,i) = sigma_points(1:6,i) + dt*X_dot;

    end

    %Media pesata dai wn
   propagated_mean(1,:) = propagated_sigma_points(1,:)*wm';
   propagated_mean(2,:) = propagated_sigma_points(2,:)*wm';
   propagated_mean(3,:) =
atan2(sin(propagated_sigma_points(3,:)*wm'),cos(propagated_sigma_points(3,:)*wm'));
   propagated_mean(4,:) =
atan2(sin(propagated_sigma_points(4,:)*wm'),cos(propagated_sigma_points(4,:)*wm'));
   propagated_mean(6,:) =
atan2(sin(propagated_sigma_points(6,:)*wm'),cos(propagated_sigma_points(6,:)*wm'));
   propagated_mean(5,:) =
atan2(sin(propagated_sigma_points(5,:)*wm'),cos(propagated_sigma_points(5,:)*wm'));

     % Tilde propagato (differenza rispetto alla media che ci servirà per
     % la matrice di covarianza campionaria)
     propagated_tilde(1,:) = propagated_sigma_points(1,:) - propagated_mean(1,:);
     propagated_tilde(2,:) = propagated_sigma_points(2,:) - propagated_mean(2,:);
     propagated_tilde(3,:) = atan2(sin(propagated_sigma_points(3,:) -
propagated_mean(3,:)),cos(propagated_sigma_points(3,:) - propagated_mean(3,:)));
     propagated_tilde(4,:) = atan2(sin(propagated_sigma_points(4,:) -
propagated_mean(4,:)),cos(propagated_sigma_points(4,:) - propagated_mean(4,:)));
     propagated_tilde(5,:) = atan2(sin(propagated_sigma_points(5,:) -
propagated_mean(5,:)),cos(propagated_sigma_points(5,:) - propagated_mean(5,:)));
     propagated_tilde(6,:) = atan2(sin(propagated_sigma_points(6,:) -
propagated_mean(6,:)),cos(propagated_sigma_points(6,:) - propagated_mean(6,:)));
```

```
    % Momento incrociato non ci serve
    propagated_cov = zeros(6,6);
    for i = 1:size(sigma_points,2)
        propagated_cov = propagated_cov +
wc(i)*propagated_tilde(:,i)*propagated_tilde(:,i)';
    end
 end



function [virtual_measurements_mean, virtual_measurements_cov, cross_cov] =
UnscentedTransform_H_Function(prior_mean, prior_cov,ht1)

    alpha = 1;
    beta = 2;
    kappa = 0;

    enne = size(prior_mean,1); % n = 6

    lambda = alpha^2*(enne + kappa) - enne;

    w0 = lambda/(lambda + enne);
    wc(1) = w0 + 1 - alpha^2 + beta;
    wc(2:2*enne+1) = 1/2/(enne+lambda);

    wm = wc;
    wm(1) = w0;

    [U,S,~] = svd(prior_cov);
    GAMMA = U*S^(1/2);

    sigma_points = prior_mean;
    for i = 1:size(GAMMA,2)
        sigma_points(:,i+1)      = prior_mean + sqrt(enne+lambda)*GAMMA(:,i);
        sigma_points(:,i+1+enne) = prior_mean - sqrt(enne+lambda)*GAMMA(:,i);
    end

    % In questo caso bisogna propagare i sigma points con il modello di
    % osservazione
    for i = 1:size(sigma_points,2)
        propagated_sigma_points(1:4,i) =
feval(ht1,sigma_points(1,i),sigma_points(2,i),sigma_points(4,i),sigma_points(5,i),0,0,0
,0);
    end


    virtual_measurements_mean(1,:) =
atan2(sin(propagated_sigma_points(1,:)*wm'),cos(propagated_sigma_points(1,:)*wm'));
    virtual_measurements_mean(2,:) =
atan2(sin(propagated_sigma_points(2,:)*wm'),cos(propagated_sigma_points(2,:)*wm'));
    virtual_measurements_mean(3,:) = propagated_sigma_points(3,:)*wm';
    virtual_measurements_mean(4,:) =
atan2(sin(propagated_sigma_points(4,:)*wm'),cos(propagated_sigma_points(4,:)*wm'));


    virtual_measurements_tilde(1,:) = atan2(sin(propagated_sigma_points(1,:) -
virtual_measurements_mean(1,:)),cos(propagated_sigma_points(1,:) -
virtual_measurements_mean(1,:)));
    virtual_measurements_tilde(2,:) = atan2(sin(propagated_sigma_points(2,:) -
virtual_measurements_mean(2,:)),cos(propagated_sigma_points(2,:) -
virtual_measurements_mean(2,:)));
```

```
    virtual_measurements_tilde(3,:) = propagated_sigma_points(3,:) -
virtual_measurements_mean(3,:);
    virtual_measurements_tilde(4,:) = atan2(sin(propagated_sigma_points(4,:) -
virtual_measurements_mean(4,:)),cos(propagated_sigma_points(4,:) -
virtual_measurements_mean(4,:)));

    virtual_measurements_cov = zeros(4,4);
    cross_cov = zeros(6,4);
    for i = 1:size(sigma_points,2)
        virtual_measurements_cov = virtual_measurements_cov +
wc(i)*virtual_measurements_tilde(:,i)*virtual_measurements_tilde(:,i)';
        cross_cov = cross_cov + wc(i)*(sigma_points(:,i) -
prior_mean)*virtual_measurements_tilde(:,i)';
    end
end
```

PLOTTINOUKF.MAT

```
figure(1)
clf
grid on
box on
title('position estimation error [m]')

log_vars.x = double(log_UKF.x_hat(:,1)');
log_vars.y = double(log_UKF.x_hat(:,2)');
log_vars.thetas = double(log_UKF.x_hat(:,3)');
log_vars.phi_dots=double(log_UKF.x_hat(:,4)');
log_vars.psis=double(log_UKF.x_hat(:,5)');
log_vars.psi_dots=double(log_UKF.x_hat(:,6)');

figure(1)
plot(log_vars.t, vecnorm([log_vars.X ; log_vars.Y] - [log_vars.x ; log_vars.y]));
hold on

figure(2)
x = log_vars.t;


tiledlayout(3,2)
nexttile
y1= log_vars.X;
y2 = log_vars.x;
hold on
plot(x,y1)
plot(x,y2)
xlabel('tempo')
ylabel('x')
legend('posizione x vera','posizione x predetta')
hold off

nexttile
y7=log_vars.phi_dot;
y8=log_vars.phi_dots;
hold on
plot(x,y7)
```

```
plot(x,y8)
xlabel('tempo')
ylabel('phi_dot')
legend('posizione phi_dot vera','posizione phi_dot predetta')
hold off

nexttile
y3=log_vars.Y;
y4=log_vars.y;
hold on
plot(x,y3)
plot(x,y4)
xlabel('tempo')
ylabel('y')
legend('posizione y vera','posizione y predetta')
hold off

nexttile
y9=log_vars.psi;
y10=log_vars.psis;
hold on
plot(x,y9)
plot(x,y10)
xlabel('tempo')
ylabel('psi')
legend('posizione psi vera','posizione psi predetta')
hold off

nexttile
y5=log_vars.theta;
y6=log_vars.thetas;
hold on
plot(x,y5)
plot(x,y6)
xlabel('tempo')
ylabel('theta')
legend('posizione theta vera','posizione theta predetta')
hold off


nexttile
y11=log_vars.psi_dot;
y12=log_vars.psi_dots;
hold on
plot(x,y11)
plot(x,y12)
xlabel('tempo')
ylabel('psi_dot')
legend('posizione psi_dot vera','posizione psi_dot predetta')
hold off

figure(3)


tiledlayout(3,2)
nexttile
y1= log_vars.X;
y2 = log_vars.x;
hold on
plot(x,y1-y2)
title('X vera - X predetta')
```

```
hold off

nexttile
y7=log_vars.phi_dot;
y8=log_vars.phi_dots;
hold on
plot(x,y7-y8)

title('phi vero - phi predetto')
hold off

nexttile
y3=log_vars.Y;
y4=log_vars.y;
hold on
plot(x,y3-y4)
title('Y vera - Y predetta')
hold off

nexttile
y9=log_vars.psi;
y10=log_vars.psis;
hold on
plot(x,y9-y10)
title('psi vero - psi predetto')
hold off

nexttile
y5=log_vars.theta;
y6=log_vars.thetas;
hold on
plot(x,y5-y6)
title('theta vero - theta predetto')
hold off


nexttile
y11=log_vars.psi_dot;
y12=log_vars.psi_dots;
hold on
plot(x,y11-y12)
title('psi_dot vero - psi_dot predetto')
hold off
```